



Instituto Universitário de Lisboa
Mestrado em Engenharia Informática

Relatório do Projeto – Map Route Explorer

1º Semestre de 2025-2026

UC: Arquitetura e Desenho de Software

Prof. José Pereira dos Reis

Alexandre Mendes nº 111026

Manuel Santos nº 111087

Ana Valente nº 111436

André Costa nº 111118

Índice

1. Introdução	7
1.1 Apresentação do Projeto	7
1.2 Objetivos	7
1.2.1 Funcionalidade e Integração	7
1.2.2 Arquitetura	8
1.2.3 Experiência de Utilizador e Design	8
1.2.4 DevOps, Qualidade e Segurança	9
2. Análise de Requisitos	9
2.1 Requisitos Funcionais	9
2.1.1 Visualização de Mapa (RF01)	9
2.1.2 Pesquisa e Geocodificação de Localizações (RF02)	10
2.1.3 Geolocalização do Utilizador (RF03)	10
2.1.4 Cálculo de Rotas Multi-Modo (RF04, RF05, RF06)	10
2.1.5 Waypoints (RF07)	10
2.1.6 Apresentação de Informações da Rota (RF08, RF09)	10
2.1.7 Pontos de Interesse ao Longo da Rota (RF10)	10
2.1.8 Auto-fit do Mapa (RF11)	11
2.1.9 Integração com Google Maps (RF12)	11
2.1.10 Chatbot Assistente (RF13)	11
2.2 Requisitos Não-Funcionais	11
2.2.1 Desempenho (RNF01)	11
2.2.2 Responsividade e Compatibilidade (RNF02, RNF06)	12
2.2.3 Disponibilidade e Escalabilidade (RNF03, RNF04)	12
2.2.4 Usabilidade (RNF05)	12
2.2.5 Segurança (RNF07)	12
2.2.6 Qualidade do código (RNF08)	12
2.2.7 Portabilidade (RNF09)	12
2.2.8 Internacionalização (RNF10)	12
2.3 Modelo Integrado de Requisitos	13
2.4 Casos Práticos	14

2.4.1 Calcular Rota Simples	14
2.4.2 Planear Rota com Waypoints	14
2.4.3 Usar Chatbot para Planeamento	14
2.4.4 Explorar Pontos de Interesse	14
2.4.5 Usar Transporte Público.....	14
3. Arquitetura do Sistema	15
3.1 Visão Geral da Arquitetura	15
3.1.1 Tipo de Sistema e Abordagem	15
3.1.2 Princípios Fundamentais da Arquitetura	15
3.1.3 Stack Tecnológico	16
3.1.4 Padrões de Design	17
3.1.5 Fluxo Arquitetural Geral	18
3.2 Diagrama C4 Completo	19
3.2.1 Nível 1 - Diagrama de Contexto	19
3.2.2 Nível 2 - Diagrama de Containers	21
3.2.3 Nível 3 - Diagrama de Componentes.....	22
3.2.4 Nível 4 - Diagrama de Código (Diagrama UML de Classes).....	24
3.3 Fundamentação das Decisões de Design	26
3.3.1 Escolha de React com TypeScript	26
3.3.2 Arquitetura SPA com Client-Side Rendering	26
3.3.3 Gestão de Estado com Zustand	26
3.3.4 Containerização com Docker	27
4. Metodologia de Desenvolvimento	28
4.1 Metodologia SCRUM	28
4.2 Planeamento do Projeto	28
4.3 User Stories e Estimativas	29
4.4 Ferramentas de Gestão do Projeto	31
4.5 Evolução do Projeto e Ajustes de Planeamento	31
5. Implementação Técnica.....	33
5.1 Módulo de Mapas	33
5.1.1 Visão Geral do Módulo de Mapas	33
5.1.2 Componente MapContainer e Configuração Base	34

5.1.3 Gestão de Limites Geográficos (BoundsEnforcer)	34
5.1.4 Gestão de Eventos e Interações no Mapa	34
5.1.5 Otimização de Atualizações e Performance.....	35
5.1.6 Camadas de Visualização Cartográfica	35
5.1.7 Ajuste Automático da Vista (AutoFitBounds).....	35
5.2 Sistema de Rotas.....	36
5.2.1 Visão Geral do Sistema de Rotas.....	36
5.2.2 Estratégia de Provedores de Routing	36
5.2.3 Integração com a Google Maps Directions API	36
5.2.4 Construção e Execução dos Pedidos de Rota	36
5.2.5 Processamento e Normalização da Resposta	36
5.2.6 Integração com o OSRM como Provedor Secundário	36
5.2.7 Suporte a Diferentes Modos de Transporte	37
5.2.8 Tratamento de Erros e Resiliência.....	37
5.3 Funcionalidade de Waypoints.....	37
5.3.1 Visão Geral da Funcionalidade de Waypoints.....	37
5.3.2 Modelo de Waypoints e Restrições	37
5.3.3 Interação do Utilizador	38
5.3.4 Processamento de Resultados e Validação	38
5.4 Integração do Chatbot.....	38
5.4.1 Visão Geral do Chatbot.....	38
5.4.2 Arquitetura da Integração	38
5.4.3 Fluxo de Comunicação	39
5.4.4 Processamento de Respostas.....	39
5.4.5 Gestão de Estado e Isolamento Funcional	39
5.5 Pontos de Interesse	39
5.5.1 Enquadramento da Funcionalidade de POIs	39
5.5.2 Integração com a Overpass API.....	39
5.5.3 Gestão e Cache de Dados	40
5.5.4 Visualização dos Pontos de Interesse	40
5.5.5 Limitação Visual e Usabilidade.....	40
5.6 Gestão de Estado	40

5.6.1 Visão Geral da Gestão de Estado	40
5.6.2 Estrutura das Stores Principais	40
5.6.3 Ações e Atualização de Estado	41
5.6.4 Integração com Componentes React	41
5.6.5 Sincronização entre Estado e APIs Externas	41
6. Qualidade de Código e CI/CD.....	42
6.1 Pipeline CI/CD	42
6.2 Ferramentas de Qualidade de Código	42
6.3 Badge de Integração Contínua	43
6.4 Dependabot	43
7. Deployment e DevOps.....	44
7.1 Arquitetura de Deployment	44
7.2 Containerização com Docker.....	45
7.3 Deploy via Portainer	46
7.4 Monitorização e Logs	46
8. Segurança.....	47
8.1 Modelo STRIDE	47
8.3 Gestão de API Keys	51
8.4 Headers de Segurança	52
9. Testes e Validação.....	53
9.1 Arquitetura de Testes.....	53
9.1.1 Fase 1: Testes de Serviços de API	53
9.1.2 Fase 2: Testes de Utilitários	55
9.1.3 Fase 3: Testes de Gestão de Estado	55
9.1.4 Fase 4: Testes de Performance.....	56
9.2 Estratégia de Mocking	56
9.2.1 Mocking de APIs HTTP	56
9.2.2 Mocking do Google Maps SDK	56
9.2.3 Mocking de APIs do Navegador	56
9.2.4 Mocking de Variáveis de Ambiente	57
9.3 Cobertura e Métricas.....	57
9.4 Validação de Integrações	57

9.4.1 Integração entre Stores e Serviços	57
9.4.2 Integração com APIs do Navegador	57
9.4.3 Integração com Serviços Externos	58
9.5 Tratamento de Erros	58
9.5.1 Erros de Rede	58
9.5.2 Erros de API	58
9.5.3 Erros de Validação.....	58
9.6 Testes de Performance	58
9.6.1 Performance de Funções Utilitárias.....	58
9.6.2 Performance em Batch	58
9.7 Execução e Integração Contínua.....	59
10. Integrações Externas.....	60
10.1 Google Maps Directions API.....	60
10.2 OSRM API	60
10.3 Google Places API	61
10.4 Overpass API.....	62
10.5 n8n Workflow	63
11. Conclusão	64

1. Introdução

1.1 Apresentação do Projeto

A realização deste projeto consiste no desenvolvimento de uma aplicação web interativa que tem o objetivo de permitir a exploração de um mapa e planeamento de rotas. Assim, o Map Route Explorer baseia-se em dados OpenStreetMap e na integração de serviços externos de cálculo de rotas e geocodificação.

Este tema foi escolhido a partir de um conjunto de propostas disponibilizadas pela unidade curricular, tendo sido selecionado o Projeto 2 (opção 2), Sistema Interativo de Rotas e Exploração de Locais com OpenStreetMap. Esta escolha deveu-se ao interesse em aplicar conceitos como integração de APIs REST, processamento de dados em formato JSON e visualização dinâmica de informação geográfica.

Desta forma, a aplicação desenvolvida deve permitir ao utilizador interagir com um mapa, selecionar pontos de origem, destino e waypoints, calcular rotas entre esses pontos e visualizar informações sobre o percurso, nomeadamente, a distância e o tempo estimado de viagem consoante o meio de transporte. Para além disso, esta aplicação suporta a integração de dados complementares, nomeadamente pontos de interesse ao longo do trajeto, de modo a melhorar a experiência de exploração do mapa.

Nota: as imagens ao longo do relatório serão enviadas noutra pasta uma vez que algumas delas são difíceis de visualizar.

1.2 Objetivos

O desenvolvimento do Map Route Explorer tem como propósito central a criação de uma aplicação web completa para planeamento de rotas, funcionando como demonstrador prático de competências em arquitetura de software. O projeto estrutura-se em quatro pilares, sendo estes a funcionalidade e integração, arquitetura, experiência do utilizador e DevOps.

1.2.1 Funcionalidade e Integração

- 1) Implementar a visualização de um mapa interativo com suporte a zoom, navegação e seleção de pontos, utilizando OpenStreetMap como base cartográfica e React-Leaflet para renderização.
- 2) Integrar a API OSRM como serviço alternativo disponível (não utilizado automaticamente no fluxo principal) para cálculo de rotas, distância e tempo de viagem para diversos meios de transporte, processando respostas em formato GeoJSON.
- 3) Integrar Google Maps Directions API como serviço primário para cálculo de rotas para todos os modos de transporte (carro, bicicleta, a pé,

transporte público), incluindo horários em tempo real para transporte público.

- 4) Implementar sistema de waypoints com limite de 5 paragens intermédias, com a reordenação manual para o cálculo otimizado de sequência.
- 5) Utilizar a Google Places API como serviço primário para geocodificação e pesquisa de localizações, sugestões em tempo real, com Nominatim como fallback.
- 6) Implementar um sistema de Points of Interest (POIs) ao longo da rota utilizando Overpass API, com filtragem por categoria (restaurantes, postos, atrações) e limite de 100 POIs por trajeto.
- 7) Desenvolver chatbot integrado com n8n workflow e Google Gemini para assistência em linguagem natural, suportando comandos em português para definição de origem, destino e waypoints.

1.2.2 Arquitetura

- 1) Aplicar princípios de arquitetura de software C4 com documentação completa de diagramas de contexto, containers, componentes e código.
- 2) Implementar a gestão de estado centralizada com Zustand para coordenar múltiplas dimensões de dados geoespaciais (mapa, rotas, pesquisa, POIs e chatbot).
- 3) Configurar build otimizado com Vite substituindo Create React App, com hot module replacement, code splitting automático e environment variables via import.meta.env.
- 4) Garantir cobertura TypeScript 100% com strict mode ativado, interfaces definidas para todos os dados de APIs e type safety em componentes React.
- 5) Implementar cache e sincronização com React Query para dados de APIs, reduzindo requisições redundantes e melhorando a performance.

1.2.3 Experiência de Utilizador e Design

- 1) Garantir interface responsiva mobile-first compatível com ecrãs de 320px a 4K, utilizando Tailwind CSS com breakpoints padronizados.
- 2) Assegurar compatibilidade cross-browser com Chrome, Firefox, Safari e Edge, testando funcionalidades específicas em cada ambiente.
- 3) Implementar feedback visual imediato durante operações assíncronas (loading states, skeleton screens, progress indicators).
- 4) Otimizar curva de aprendizagem para <30 segundos para tarefas básicas (definir origem/destino, calcular rota) através de UI intuitiva e fluxos simplificados.
- 5) Implementar auto-fit do mapa ajustando automaticamente zoom e bounds para mostrar toda a rota com margens adequadas.

1.2.4 DevOps, Qualidade e Segurança

- 1) Configurar containerização completa com Docker multi-stage builds, docker-compose para orquestração, e Nginx como reverse proxy.
- 2) Implementar pipeline CI (Integração Contínua) com GitHub Actions executando lint, build, TypeScript check em cada pull request. O deploy é realizado manualmente via Portainer.
- 3) Configurar deploy via Portainer com health checks e gestão visual de containers.
- 4) Validar performance do sistema, mantendo o tempo de resposta menor que 3 segundos para 95% dos cálculos de rota, com monitorização via logs e métricas.
- 5) Implementar mecanismos de segurança incluindo CORS headers configurados no Nginx, security headers (X-Frame-Options, X-Content-Type-Options, etc.) e API keys injetadas em build-time.
- 6) Garantir disponibilidade de 99% em ambiente de produção com health checks configurados, restart policies, e fallback entre APIs.

2. Análise de Requisitos

2.1 Requisitos Funcionais

Os requisitos funcionais do sistema foram definidos com base na proposta do projeto disponibilizada pela unidade curricular, juntamente com as funcionalidades adicionais identificadas ao longo do desenvolvimento através de metodologia ágil Scrum. Desta forma, estes requisitos descrevem as funcionalidades que o sistema deve disponibilizar ao utilizador, garantindo o cumprimento dos objetivos estabelecidos para o Map Route Explorer como ferramenta de planeamento de rotas multi-modo.

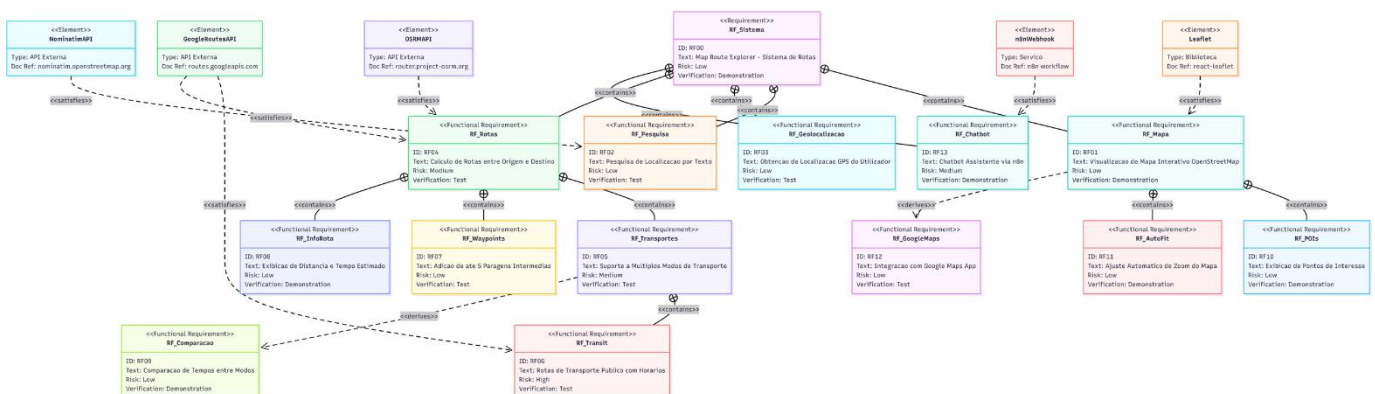


Figura 1 - Diagrama de Requisitos Funcionais

2.1.1 Visualização de Mapa (RF01)

O sistema deve permitir a visualização de um mapa interativo baseado em dados do OpenStreetMap, disponibilizando funcionalidades essenciais de navegação,

tais como zoom, pan e seleção de pontos através de interação direta com o mapa. A visualização inicial deve centrar-se em Lisboa (38.7223, -9.1393) com zoom nível 13 por default.

2.1.2 Pesquisa e Geocodificação de Localizações (RF02)

O sistema deve disponibilizar um campo de pesquisa que permita ao utilizador procurar endereços e localizações através da Google Places API como serviço primário (e com Nominatim como fallback), com sugestões em tempo real (autocomplete) após mínimo de 3 caracteres e debouncing de 500ms para otimização de requisições.

2.1.3 Geolocalização do Utilizador (RF03)

O sistema deve obter automaticamente a localização atual do utilizador via API de geolocalização do navegador, após consentimento explícito, e disponibilizar um botão "Usar minha localização" para definição rápida do ponto de partida.

2.1.4 Cálculo de Rotas Multi-Modo (RF04, RF05, RF06)

O sistema deve calcular rotas entre origem e destino, suportando quatro modos de transporte, sendo estes carro, bicicleta, a pé e transporte público, utilizando Google Maps Directions API como serviço primário para todos os modos. O OSRM está disponível como fallback para modos individuais (carro, bicicleta, a pé).

2.1.5 Waypoints (RF07)

O sistema deve permitir a definição de até 5 paragens intermédias (waypoints) entre o ponto de origem e o destino final, construindo dinamicamente a requisição às APIs de roteamento com múltiplos pontos e permitindo adição e remoção de waypoints (reordenação manual removendo e readicionando).

2.1.6 Apresentação de Informações da Rota (RF08, RF09)

Após o cálculo da rota, o sistema deve apresentar informações relevantes ao utilizador, nomeadamente a distância total em quilómetros e o tempo estimado de viagem em formato legível (ex: 1h 30min), permitindo ainda comparação de tempos entre diferentes modos de transporte.

2.1.7 Pontos de Interesse ao Longo da Rota (RF10)

O sistema deve permitir a obtenção e visualização de pontos de interesse (POIs) ao longo da rota calculada, recorrendo à API Overpass para categorias como restaurantes, postos de combustível, estacionamento e atrações turísticas, com limite de até 100 POIs retornados pela API, sendo exibidos até 30 no mapa.

2.1.8 Auto-fit do Mapa (RF11)

O sistema deve ajustar automaticamente o nível de zoom e os limites do mapa para exibir toda a rota calculada, incluindo origem, destino e waypoints, com margens adequadas para uma visualização otimizada.

2.1.9 Integração com Google Maps (RF12)

O sistema deve permitir ao utilizador abrir a rota calculada diretamente na aplicação Google Maps ao clicar no ícone das informações de rota, incluindo todos os waypoints definidos para navegação em tempo real.

2.1.10 Chatbot Assistente (RF13)

O sistema deve integrar um chatbot para assistência ao utilizador através de workflow n8n com processamento de linguagem natural via Google Gemini e o uso da Google Places API para geocoding, permitindo uma interação em português para definição de rotas por comando de texto.

2.2 Requisitos Não-Funcionais

Os requisitos não-funcionais definem os atributos de qualidade do sistema, assegurando que a solução não apenas funciona corretamente, mas também oferece uma experiência de utilização eficiente, segura e escalável.

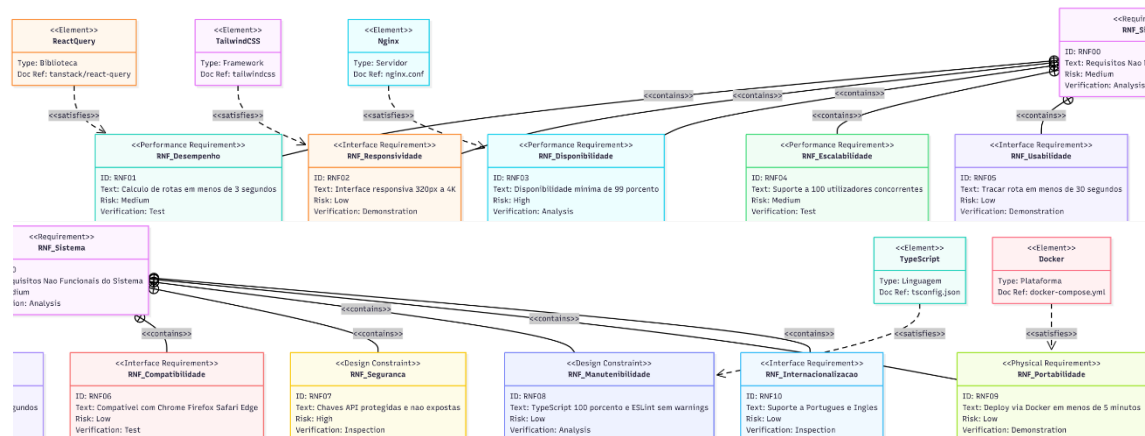


Figura 2 - Diagrama de Requisitos Não Funcionais

2.2.1 Desempenho (RNF01)

O sistema deve calcular rotas em menos de 3 segundos para 95% das requisições, com tempo médio de resposta de 1.5-2.5 segundos para rotas com localização atual e 2-3 segundos para rotas completas.

2.2.2 Responsividade e Compatibilidade (RNF02, RNF06)

A interface deve ser totalmente responsiva e funcional em dispositivos móveis (a partir de 320px) até monitores 4K, com compatibilidade garantida para Chrome, Firefox, Safari e Edge nas duas últimas versões principais.

2.2.3 Disponibilidade e Escalabilidade (RNF03, RNF04)

O sistema deve manter disponibilidade mínima de 99% em ambiente de produção e suportar pelo menos 100 utilizadores concorrentes através de arquitetura stateless e containerização Docker.

2.2.4 Usabilidade (RNF05)

A interface deve permitir que um utilizador médio trace uma rota completa (origem + destino + cálculo) em menos de 30 segundos, seguindo princípios de design intuitivo e feedback visual imediato.

2.2.5 Segurança (RNF07)

As chaves de API (Google Maps, Google Places) devem ser protegidas através de injeção em tempo de build, nunca expostas em repositório público, com restrições de domínio configuradas nos respetivos painéis de controlo.

2.2.6 Qualidade do código (RNF08)

O código deve seguir padrões de qualidade estabelecidos, com cobertura TypeScript de 100%, zero warnings no ESLint, e documentação adequada seguindo padrão C4 para arquitetura.

2.2.7 Portabilidade (RNF09)

O sistema deve ser facilmente deployable em qualquer ambiente compatível com Docker, com processo de deploy completo executável em menos de 5 minutos usando docker-compose.

2.2.8 Internacionalização (RNF10)

O sistema deve suportar múltiplos idiomas, começando com português como língua primária e inglês como secundária, com estrutura preparada para futuras expansões linguísticas.

2.3 Modelo Integrado de Requisitos

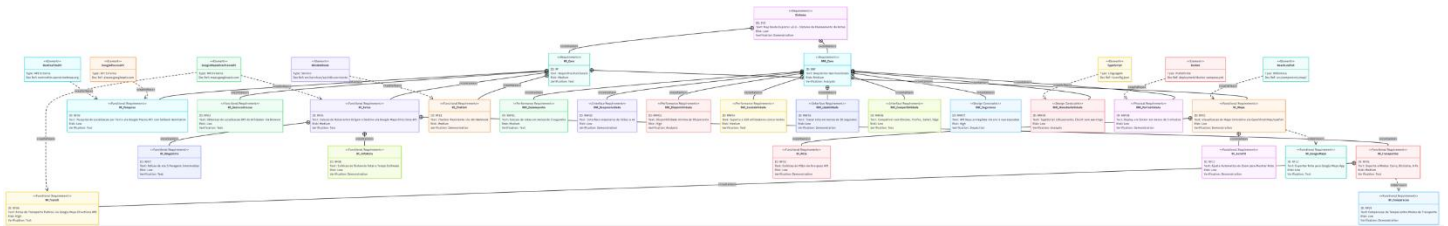


Figura 3 - Diagrama geral dos RF e RNF com as componentes do sistema

A Figura 3 representa o modelo completo de requisitos do sistema Map Route Explorer, evidenciando a relação direta entre os requisitos identificados e os elementos concretos de implementação que os satisfazem. O diagrama organiza os requisitos de forma hierárquica, distinguindo claramente entre requisitos funcionais e requisitos não funcionais, bem como as dependências e derivações existentes entre eles.

Os requisitos funcionais descrevem as capacidades essenciais do sistema, centradas no planeamento e exploração de rotas. Estes incluem a visualização de mapas interativos, a pesquisa e obtenção de localizações, o cálculo de rotas com diferentes modos de transporte, a gestão de pontos intermédios (waypoints), a apresentação de informação detalhada das rotas, a integração de pontos de interesse e a disponibilização de um chatbot assistente. O diagrama evidencia ainda relações de composição e derivação entre requisitos, como a forma como o cálculo de rotas agrega suporte a múltiplos modos de transporte e funcionalidades associadas. Cada requisito funcional é ligado explicitamente aos componentes técnicos que o concretizam, como a biblioteca React-Leaflet para visualização cartográfica, as APIs da Google e do OpenStreetMap para geocodificação e routing, e o serviço n8n para suporte ao chatbot.

Os requisitos não funcionais definem atributos de qualidade e restrições de design que condicionam o comportamento global do sistema. Estes requisitos abrangem aspetos como desempenho, responsividade da interface, disponibilidade, escalabilidade, usabilidade, compatibilidade entre navegadores, segurança das credenciais, manutenibilidade do código e portabilidade da solução. No diagrama, cada requisito não funcional encontra-se associado a um nível de risco e a um método de verificação específico, reforçando a sua rastreabilidade e mensurabilidade. Elementos como Docker e TypeScript surgem explicitamente associados ao cumprimento dos requisitos de portabilidade e manutenibilidade, respetivamente.

2.4 Casos Práticos

2.4.1 Calcular Rota Simples

O utilizador, após carregar a aplicação e visualizar o mapa, define um ponto de origem e um destino, seja através de clique direto no mapa ou utilizando a funcionalidade de pesquisa. O sistema processa o pedido, calcula a rota correspondente e apresenta-a visualmente no mapa, juntamente com a distância total e o tempo estimado de viagem. O utilizador tem ainda a possibilidade de alternar entre diferentes modos de transporte, como carro, bicicleta, a pé ou transporte público, com o sistema a recalculá-la automaticamente para cada opção.

2.4.2 Planear Rota com Waypoints

Para planear uma rota com múltiplas paragens intermédias, o utilizador, após definir a origem e o destino, interage com a interface para adicionar waypoints, até um máximo de cinco. O sistema recalcula a rota de forma a incluir todas as paragens definidas. A interface permite a reordenação destes pontos manualmente, ou seja, o utilizador retira os pontos e volta a pôr novos ou os mesmos para perceber qual é a ordem mais eficiente (com a ajuda da interface que mostra o tempo e distância do percurso).

2.4.3 Usar Chatbot para Planeamento

O utilizador que prefere uma interação mais natural pode recorrer ao chatbot integrado. Ao abrir o widget de conversa e enviar uma mensagem em linguagem natural, como "Quero ir de Lisboa ao Porto", o sistema encaminha o pedido para o workflow configurado no n8n. Este, por sua vez, processa a intenção com o Gemini e obtém coordenadas via Google Places API (com fallback para Nominatim). A rota pode ser automaticamente aplicada na interface quando o n8n retorna uma ação explícita e o chatbot confirma a ação. Em outros casos, o chatbot atua como assistente da aplicação.

2.4.4 Explorar Pontos de Interesse

Com uma rota já calculada, um utilizador pode ativar o filtro de Pontos de Interesse (POIs). O sistema consulta então a Overpass API para obter locais relevantes ao longo do trajeto, como restaurantes ou monumentos, e exibe-os no mapa através de ícones categorizados. Ao clicar num POI, o utilizador acede a detalhes sobre o local (nome e categoria).

2.4.5 Usar Transporte Público

Quando o utilizador seleciona o modo Transporte Público, o sistema consulta a Google Maps Directions API para obter rotas otimizadas com base em horários reais. A rota calculada é apresentada no mapa, e o sistema mostra o tempo total

estimado de viagem. Esta opção pode ser comparada com os tempos dos outros modos de transporte disponíveis, permitindo uma decisão informada.

3. Arquitetura do Sistema

3.1 Visão Geral da Arquitetura

3.1.1 Tipo de Sistema e Abordagem

O Map Route Explorer é uma aplicação web moderna baseada numa Single Page Application (SPA), seguindo uma arquitetura client-side rendering onde a maior parte do processamento ocorre no navegador do utilizador. Esta escolha permite uma experiência de utilização fluída e responsiva, essencial para aplicações interativas de mapas que requerem atualizações frequentes da interface sem recarregamentos completos da página.

3.1.2 Princípios Fundamentais da Arquitetura

1) Separação de Responsabilidades

A arquitetura do sistema estabelece uma clara separação de responsabilidades através de quatro camadas distintas. A camada de apresentação, que é implementada com componentes React, é responsável pela interface do utilizador e renderização visual. A camada de lógica é gerida através de stores Zustand e hooks React, sendo possível assim coordenar o estado da aplicação e as regras de domínio. A camada de serviços estabelece a comunicação com APIs externas através de módulos especializados, enquanto que a camada de infraestrutura, baseada em Docker e Nginx, assegura a containerização e o proxy reverso do sistema.

2) Design Stateless

O sistema não mantém estado no servidor, seguindo o princípio stateless que facilita a escalabilidade horizontal. Todo o estado da aplicação (rotas definidas, posição do mapa, modo de transporte) é gerido no cliente através do Zustand, com persistência opcional via localStorage para preferências do utilizador.

3) Abordagem API-First

O desenvolvimento seguiu uma filosofia API-first, onde a integração com serviços externos foi considerada desde o início. O sistema atua como um orquestrador de múltiplas APIs especializadas, cada uma fornecendo

funcionalidades específicas: geocodificação (Google Places), cálculo de rotas (Google Maps Directions), pontos de interesse (Overpass), e processamento de linguagem natural (n8n com Gemini).

4) Containerização e Portabilidade

Através da containerização com Docker, a aplicação é empacotada como uma unidade independente que pode ser executada consistentemente em qualquer ambiente, desde desenvolvimento local até produção. Este isolamento garante que dependências e configurações sejam reproduzíveis e controladas.

3.1.3 Stack Tecnológico

A tabela seguinte resume o stack tecnológico selecionado para cada camada da arquitetura:

Camada	Tecnologia	Propósito
Frontend	React 18 + TypeScript	Interface do utilizador
Build Tool	Vite 5	Bundling e dev server
State Management	Zustand	Estado global da aplicação
Data Fetching	React Query	Cache e sincronização
Styling	Tailwind CSS	UI responsiva
Maps	Leaflet + React Leaflet	Renderização de mapas
Web Server	Nginx Alpine	Servir assets e proxy
Container	Docker	Deployment e isolamento

3.1.4 Padrões de Design

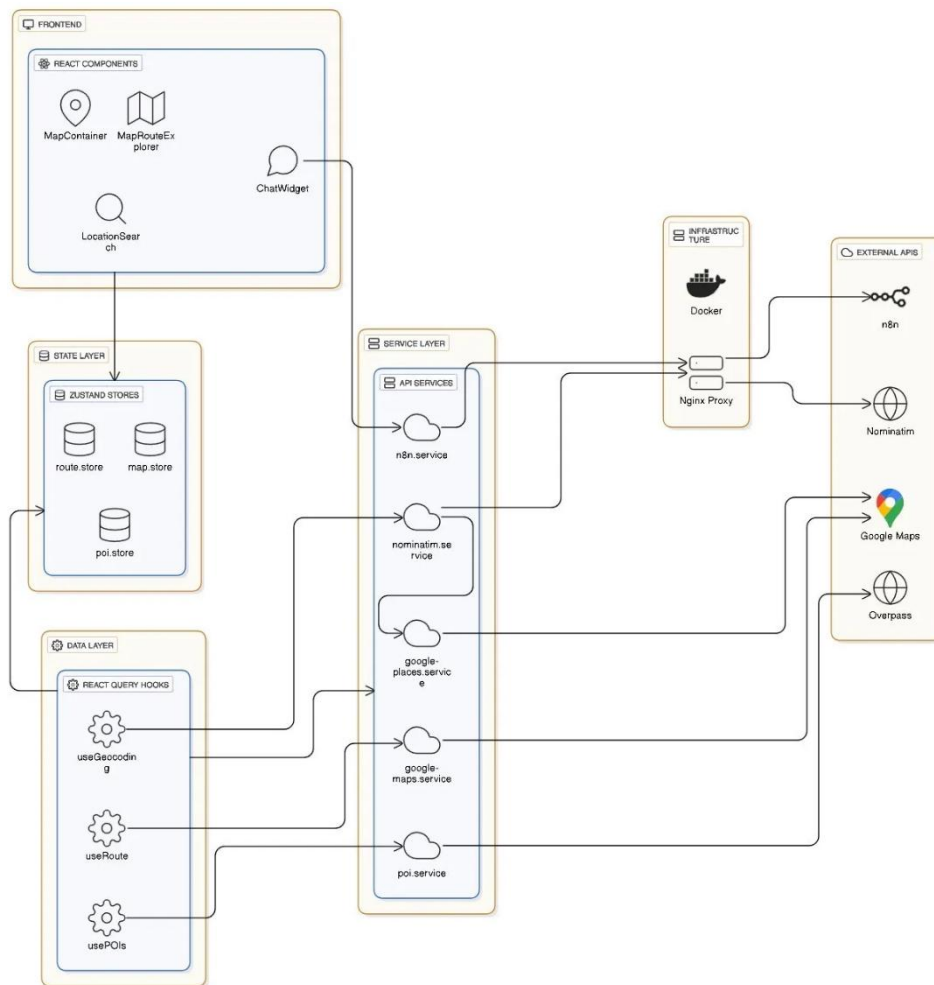


Figura 4 - Visão arquitetural da aplicação Map Route Explorer

1) Component-Based Architecture

A interface da aplicação é construída seguindo uma arquitetura baseada em componentes, claramente visível na camada de frontend da Figura 4. Esta abordagem decompõe a interface em elementos reutilizáveis e independentes como o MapContainer, MapRouteExplorer, LocationSearch e ChatWidget. Os componentes organizam-se conforme a sua responsabilidade, existindo componentes de apresentação que apenas renderizam elementos visuais, componentes de container que coordenam estado e lógica, e componentes especializados que encapsulam funcionalidades específicas.

2) Service Layer Pattern

A comunicação com serviços externos é uniformizada através de uma camada de serviços, representada na secção de API Services. Esta camada atua como abstração para todas as APIs externas, incluindo os serviços de geocodificação do Nominatim, os mapas da Google e o serviço de pontos de

interesse. Com a centralização da configuração, o tratamento de erros e a lógica de chamadas, assegura um ponto único de contacto com o exterior, simplificando manutenção, testes e eventual substituição de provedores sem afetar o resto da aplicação.

3) State Management Pattern com Zustand

A gestão do estado da aplicação implementa o padrão store através da biblioteca Zustand, conforme ilustrado na camada de estado. Foram criadas várias stores especializadas para domínios distintos sendo estas a route.store para estado das rotas, map.store para estado da visualização do mapa e search.store para gerir pesquisas de localizações e POIStore para gerir pontos de interesse. Esta divisão garante uma gestão de estado modular e previsível, com atualizações reativas em toda a aplicação.

4) Repository Pattern via React Hooks

O acesso e manipulação de dados geográficos são abstraídos através de custom React hooks, que funcionam como implementação do repository pattern. Estes hooks, como useGeocoding, useRoute e usePOIs, isolam completamente a lógica de obtenção de dados (seja de uma API REST ou de outra fonte) da sua utilização nos componentes de interface. Esta separação proporciona uma camada de dados limpa e intercambiável, facilitando testes e manutenção.

3.1.5 Fluxo Arquitetural Geral

O fluxo de dados no Map Route Explorer segue uma abordagem unidirecional e em camadas (Figura 4). As interações iniciam-se na camada de apresentação, onde os componentes React capturam as ações do utilizador, seja através de cliques no mapa, preenchimento de campos de pesquisa ou seleção de opções de transporte. Estas ações são então processadas pela camada de gestão de estado, implementada através de stores Zustand especializadas, que atualizam o estado interno da aplicação de forma previsível e consistente.

A comunicação com serviços externos é mediada por uma camada de serviços que atua como abstração para as diversas APIs integradas no sistema. Esta camada centraliza a configuração, o tratamento de erros e a transformação de dados, garantindo que o resto da aplicação permaneça isolado das especificidades de cada provedor externo. Os fluxos de comunicação específicos, detalhados no Capítulo 10, ilustram como estas interações ocorrem em cenários concretos como o cálculo de rotas ou a pesquisa de localizações.

As respostas dos serviços externos são processadas e utilizadas para atualizar novamente o estado da aplicação, desencadeando uma re-renderização dos componentes afetados. Este ciclo completo, da interação do utilizador até à atualização da interface ocorre de forma fluída graças à arquitetura reativa do React combinada com a gestão de estado eficiente do Zustand. A organização em camadas independentes, com interfaces bem definidas entre elas, permite que cada componente do sistema possa ser desenvolvido, testado e mantido de forma isolada, enquanto a integração geral mantém-se coesa e previsível.

Esta abordagem arquitetural garante escalabilidade ao permitir a adição de novas funcionalidades através da extensão das camadas existentes ou da criação de novas, sem perturbar o funcionamento estabelecido. A clara separação entre lógica de interface, gestão de estado, comunicação externa e renderização visual estabelece uma base sólida para o desenvolvimento contínuo e evolução da aplicação.

3.2 Diagrama C4 Completo

3.2.1 Nível 1 - Diagrama de Contexto

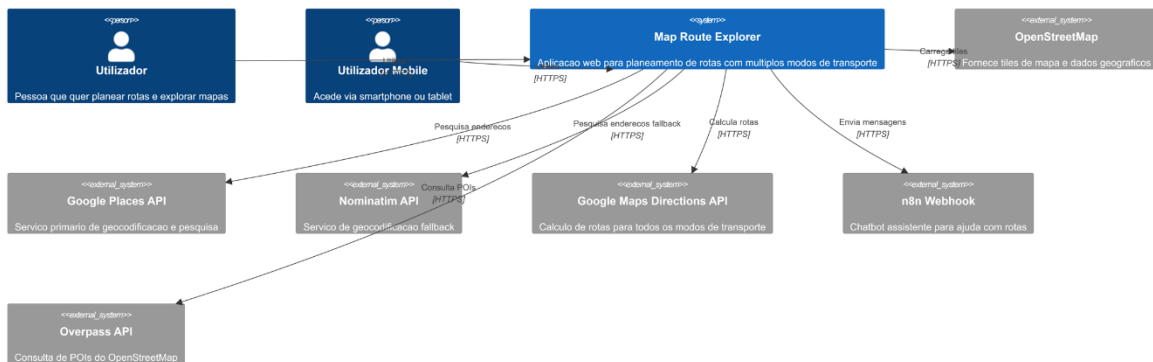


Figura 5 - Diagrama de Contexto (C4 Level 1)

O Diagrama de Contexto C4, apresentado na Figura 5, estabelece a visão mais abrangente do sistema Map Route Explorer e do seu ecossistema externo. Este diagrama situa a aplicação no centro das suas interações com os diferentes utilizadores e sistemas com os quais comunica.

O sistema principal (Map Route Explorer) é caracterizado como uma aplicação web dedicada ao planeamento de rotas com suporte a múltiplos modos de transporte. Interage diretamente com dois tipos de utilizadores, sendo estes o “Utilizador”, que acede à aplicação através de um navegador web convencional num ambiente com desktop/laptop e o “Utilizador Mobile”, que utiliza dispositivos

móveis como smartphones ou tablets. Em ambos os casos, a comunicação é efetuada através do protocolo HTTPS, garantindo a confidencialidade e integridade dos dados trocados.

O diagrama identifica seis sistemas externos essenciais para o funcionamento da aplicação. O OpenStreetMap fornece os tiles de mapa e dados geográficos necessários para a visualização cartográfica. A Google Places API é utilizada como serviço primário de geocodificação e pesquisa de endereços, permitindo converter descrições textuais introduzidas pelo utilizador em localizações geográficas precisas. Como mecanismo de redundância e resiliência, a Nominatim API é integrada como serviço de geocodificação alternativo, sendo utilizada manualmente como fallback sempre que o serviço primário não se encontre disponível.

O cálculo de rotas é feito pela Google Maps Directions API, que suporta diferentes modos de transporte, incluindo transporte individual e transporte público. A integração com o n8n Webhook viabiliza a funcionalidade de chatbot assistente, responsável por processar mensagens dos utilizadores e fornecer apoio ao planeamento de rotas. Por fim, a Overpass API é utilizada para a consulta de Points of Interest (POIs) com base nos dados do OpenStreetMap, enriquecendo a experiência de exploração ao longo das rotas.

Todas as comunicações entre o Map Route Explorer e os sistemas externos utilizam exclusivamente o protocolo HTTPS, assegurando a encriptação dos dados em trânsito e protegendo a aplicação contra ataques de intercepção.

Este diagrama de contexto constitui o ponto de partida para a compreensão global do sistema, definindo claramente os limites entre componentes internos e dependências externas, bem como as integrações críticas que suportam as funcionalidades disponibilizadas aos utilizadores.

3.2.2 Nível 2 - Diagrama de Containers

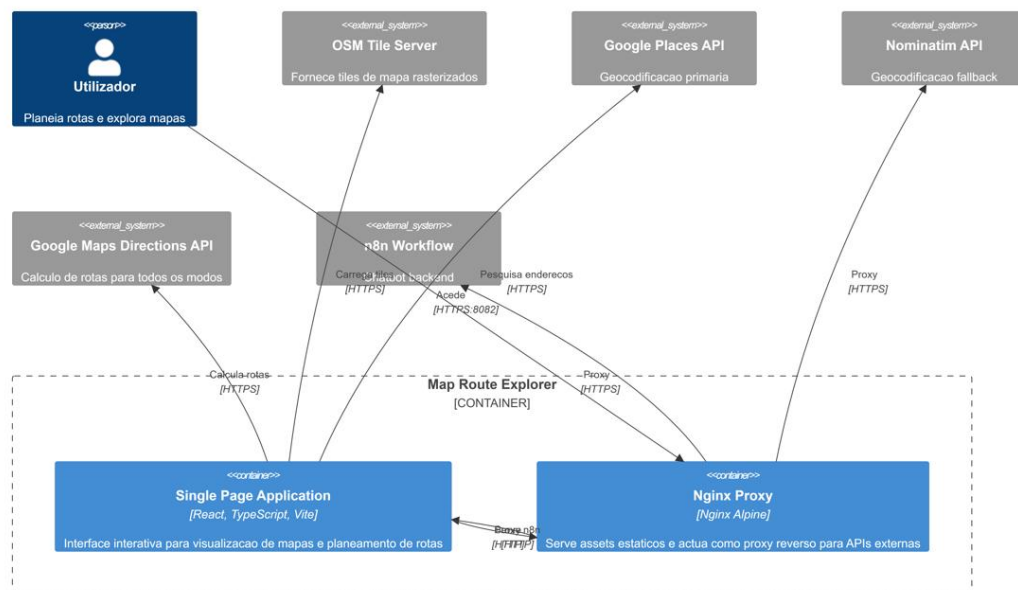


Figura 6 - Diagrama de Containers (C4 Level 2)

O Diagrama de Containers C4, apresentado na Figura 6, descreve a estrutura interna do sistema Map Route Explorer ao nível dos seus principais componentes executáveis e das interações com serviços externos. Este nível evidencia como a solução se organiza em termos de containers, responsabilidades e fluxos de comunicação, servindo de ponte entre a visão de contexto e os detalhes de implementação.

Dentro do limite do sistema “Map Route Explorer”, o diagrama identifica dois componentes lógicos principais. A Single Page Application, desenvolvida com React, TypeScript e Vite, corresponde ao frontend da aplicação e é disponibilizada sob a forma de ficheiros estáticos. Estes ficheiros são servidos por um container baseado em Nginx Alpine, que constitui o único componente executável do sistema em ambiente de produção. O Nginx desempenha simultaneamente o papel de servidor web para os assets da aplicação e de proxy reverso para determinados serviços externos.

O acesso dos utilizadores ao sistema é efetuado através do Nginx Proxy, na porta 8082 (mapeada para porta 80 interna). O HTTPS é tratado por um proxy reverso externo. Internamente, a comunicação entre o Nginx e a aplicação cliente ocorre através de HTTP, uma vez que se trata de tráfego local dentro do mesmo ambiente containerizado, não exposto ao exterior.

O diagrama evidencia dois padrões distintos de comunicação com sistemas externos. Por um lado, a Single Page Application comunica diretamente via HTTPS com serviços externos públicos, nomeadamente o OSM Tile Server,

responsável pelo fornecimento dos tiles de mapa, a Google Places API, utilizada como serviço primário de geocodificação e pesquisa de endereços, e a Google Maps Directions API, responsável pelo cálculo de rotas para diferentes modos de transporte.

Por outro lado, determinados serviços são acedidos exclusivamente através do proxy Nginx. As requisições à Nominatim API, utilizada como mecanismo de fallback para geocodificação, e ao n8n Workflow, responsável pelo backend do chatbot assistente, são encaminhadas primeiro para o Nginx através de HTTP interno e posteriormente reenviadas para os serviços externos via HTTPS. Esta estratégia permite centralizar preocupações transversais como controlo de CORS, aplicação de políticas de rate limiting, adição de cabeçalhos específicos e registo de logs, sem sobrecarregar a lógica da aplicação cliente.

Esta arquitetura baseada em containers promove uma separação clara de responsabilidades, ao mesmo tempo que mantém uma estrutura simples e eficiente.

3.2.3 Nível 3 - Diagrama de Componentes

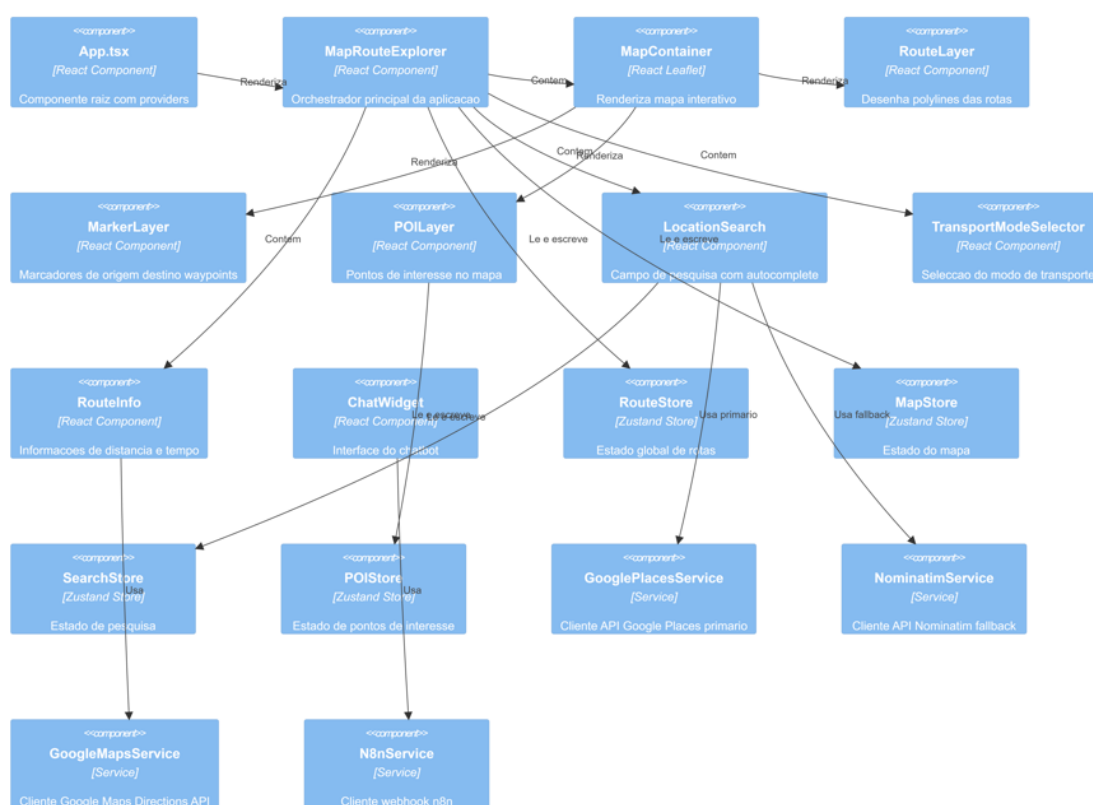


Figura 7 - Diagrama de Componentes (C4 Level 3)

O Diagrama de Componentes C4, apresentado na Figura 7, fornece uma visão detalhada da estrutura interna da Single Page Application React, evidenciando a decomposição lógica da aplicação em componentes de interface, stores de

estado e serviços de integração externa. Este nível de detalhe permite compreender como a aplicação organiza as suas responsabilidades internas e como os diferentes elementos colaboram para suportar as funcionalidades disponibilizadas ao utilizador.

No topo da hierarquia encontra-se o componente `App.tsx`, que atua como componente raiz da aplicação. Este componente é responsável por inicializar a aplicação React e configurar os providers globais necessários, nomeadamente os relacionados com gestão de estado e contexto. A partir deste ponto, é renderizado o componente `MapRouteExplorer`, que assume o papel de orquestrador principal da aplicação. Este componente coordena os diversos elementos funcionais, gere o fluxo de dados entre interface e estado global e garante a consistência das interações do utilizador.

A camada de visualização cartográfica é composta por um conjunto de componentes especializados e fortemente coesos. O `MapContainer`, baseado em `React Leaflet`, é responsável pela renderização do mapa interativo, constituindo a base visual da aplicação. Sobre este componente são renderizadas camadas adicionais, nomeadamente o `RouteLayer`, que desenha as *polylines* correspondentes às rotas calculadas, o `MarkerLayer`, que apresenta marcadores para origem, destino e waypoints, e o `POILayer`, que visualiza os pontos de interesse obtidos a partir de fontes externas.

A interface de interação com o utilizador é composta por vários componentes dedicados a funcionalidades específicas. O `LocationSearch` implementa o campo de pesquisa com suporte a *autocomplete*, permitindo ao utilizador pesquisar localizações de forma assistida. O `TransportModeSelector` possibilita a seleção do modo de transporte pretendido. O `RouteInfo` apresenta informações detalhadas sobre as rotas calculadas, como distância e tempo estimado. O `ChatWidget` fornece a interface de comunicação com o chatbot assistente, integrando funcionalidades conversacionais na aplicação.

A gestão de estado global é realizada através de quatro stores `Zustand`, cada uma dedicada a um domínio funcional específico. O `RouteStore` mantém toda a informação relacionada com o planeamento de rotas, incluindo origem, destino, waypoints e resultados do cálculo. O `MapStore` gere o estado associado à visualização do mapa, como centro, nível de zoom e interações do utilizador. O `SearchStore` é responsável pelo estado da pesquisa de localizações, incluindo o texto introduzido, resultados e estados de carregamento. O `POIStore` gere os dados relativos a pontos de interesse, incluindo listas de POIs, filtros ativos e estado de carregamento.

A comunicação com serviços externos é abstraída através de uma camada de serviços dedicada. O `GooglePlacesService` atua como cliente primário para a `Google Places API`, fornecendo funcionalidades de geocodificação e pesquisa de localizações. O `NominatimService` é utilizado como mecanismo de *fallback*

quando necessário. O GoogleMapsService encapsula a comunicação com a Google Maps Directions API para cálculo de rotas. O N8nService gere a interação com o webhook do chatbot. Cada um destes serviços centraliza a lógica específica da API correspondente, incluindo inicialização, configuração, transformação de dados e tratamento de erros.

Os componentes de interface consomem estes serviços de forma indireta, mantendo uma separação clara entre lógica de apresentação, estado e integração externa. O MapRouteExplorer atua como mediador entre os componentes visuais e as stores de estado, assegurando que alterações de estado resultam em atualizações consistentes da interface.

3.2.4 Nível 4 - Diagrama de Código (Diagrama UML de Classes)

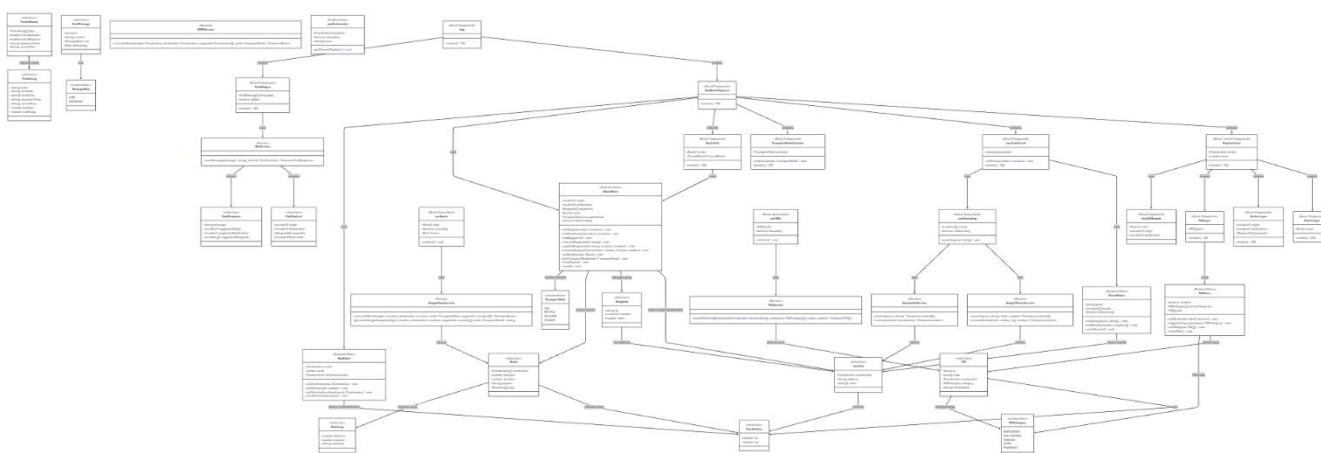


Figura 8 - Diagrama de Classes (C4 Level 4)

O Diagrama de Classes C4, apresentado na Figura 8, representa o nível mais detalhado da arquitetura do Map Route Explorer, descrevendo explicitamente os modelos de dados, estruturas de estado, serviços de integração e relações entre componentes que suportam o comportamento interno da aplicação. Este nível foca-se na organização lógica do domínio e nas interações entre entidades, permitindo compreender como a informação é estruturada, partilhada e manipulada ao longo do sistema.

No núcleo do modelo encontram-se as interfaces de domínio fundamentais, responsáveis por representar conceitos geográficos e funcionais centrais. As interfaces `Coordinates`, `Location` e `Waypoint` estabelecem a base para a representação de pontos geográficos, distinguindo entre coordenadas puras e localizações enriquecidas com metadados como nome e endereço. A separação explícita entre `Coordinates` e `Location` permite flexibilidade na utilização dos dados, possibilitando operações de routing e visualização mesmo quando apenas coordenadas estão disponíveis, sem dependência obrigatória de informação textual.

A entidade `Route` modela uma rota completa calculada pelo sistema, agregando geometria, distância total, duração estimada e uma decomposição em segmentos através da interface `RouteLeg`. Esta estrutura reflete diretamente o formato devolvido pelos serviços de cálculo de rotas externos, ao mesmo tempo que fornece uma representação normalizada utilizada internamente pela aplicação. O sistema suporta transporte público através da interface `Route` genérica, utilizando o modo de transporte `TransportMode.TRANSIT`, que captura informação específica como paragens, linhas, cores e número de paragens, refletindo a complexidade adicional deste tipo de percurso.

O suporte a diferentes modos de transporte é formalizado através da enumeração `TransportMode`, prevenindo estados inválidos e garantindo consistência em toda a aplicação. A funcionalidade de pontos de interesse é suportada pelas entidades `POI` e `POICategory`, que permitem classificar, filtrar e representar elementos relevantes ao longo da rota. Esta associação explícita facilita tanto a renderização visual diferenciada como a aplicação de filtros dinâmicos definidos pelo utilizador.

No domínio da interação conversacional, o modelo inclui as entidades `ChatMessage`, `ChatContext` e `ChatResponse`. Estas estruturas formalizam a comunicação com o chatbot assistente, encapsulando não apenas o conteúdo textual das mensagens, mas também o contexto geográfico e funcional da aplicação no momento da interação. Esta abordagem permite que o chatbot forneça respostas contextualizadas, incluindo sugestões diretas de origem, destino ou waypoints, integrando-se de forma natural no fluxo de planeamento de rotas.

A gestão de estado da aplicação é estruturada através de várias stores `Zustand` especializadas, cada uma responsável por um domínio funcional distinto. O `RouteStore` centraliza toda a informação relacionada com o planeamento de rotas, incluindo origem, destino, waypoints, rota calculada e modo de transporte selecionado. O `MapStore` gere o estado da visualização cartográfica, como centro, nível de zoom e localizações selecionadas. O `POIStore` é responsável pelo estado dos pontos de interesse, incluindo categorias ativas e listas de POIs, enquanto o `SearchStore` gere o estado associado à pesquisa de localizações, como o texto introduzido, resultados e estados de carregamento. Esta separação clara reforça o princípio de responsabilidade única e reduz o acoplamento entre funcionalidades.

A comunicação com sistemas externos é abstraída através de serviços dedicados. O `GooglePlacesService` fornece funcionalidades de pesquisa e geocodificação direta, sendo complementado pelo `NominatimService` como mecanismo de fallback. O `GoogleMapsService` encapsula a integração com a Google Maps Directions API para cálculo de rotas e geração de URLs externas.

O POIService é responsável pela obtenção de pontos de interesse ao longo das rotas, enquanto o N8nService gere a comunicação com o backend do chatbot.

O acesso a dados assíncronos é operacionalizado através de React Hooks especializados, como useRoute, useGeocoding, useGeolocation e usePOIs. Estes hooks atuam como intermediários entre os serviços externos e os componentes React, encapsulando lógica de carregamento, tratamento de erros e sincronização com o estado global. Esta abordagem promove reutilização, testabilidade e consistência no consumo de dados ao longo da aplicação.

3.3 Fundamentação das Decisões de Design

3.3.1 Escolha de React com TypeScript

A seleção do React combinado com TypeScript fundamentou-se na necessidade de type safety robusta para a manipulação de dados geoespaciais complexos que o sistema processa. Coordenadas geográficas, estruturas de rotas com múltiplos waypoints, e respostas de APIs externas exigem validação rigorosa de tipos que o TypeScript providencia nativamente. Esta combinação permite detetar erros em tempo de compilação, reduzindo bugs relacionados com a manipulação incorreta de estruturas de dados. Adicionalmente, a arquitetura baseada em componentes do React demonstra-se particularmente adequada para interfaces de mapa, onde elementos como marcadores, layers de rota e controles de zoom beneficiam de encapsulamento e reutilização. O ecossistema React oferece bibliotecas maduras para a integração com sistemas de mapas, nomeadamente o React-Leaflet que simplifica significativamente a renderização de mapas interativos dentro do paradigma de componentes.

3.3.2 Arquitetura SPA com Client-Side Rendering

A opção por uma Single Page Application (SPA) com renderização no cliente justifica-se pelos requisitos de experiência de utilizador específicos a aplicações de mapas interativos. Operações frequentes como zoom, pan, seleção de pontos e alternância entre modos de transporte necessitam de resposta imediata sem recarregamentos completos de página que interromperiam a experiência de utilização. O client-side rendering permite que estes updates ocorram de forma fluída, mantendo o estado da aplicação no navegador. Esta abordagem também possibilita a implementação eficiente de cache local tanto para tiles de mapa como para resultados de rotas previamente calculadas, reduzindo chamadas redundantes a serviços externos e melhorando significativamente os tempos de resposta para utilizadores que revisitam as mesmas áreas ou rotas.

3.3.3 Gestão de Estado com Zustand

A gestão do estado complexo do Map Route Explorer, que inclui múltiplas dimensões interrelacionadas (localizações, rotas, modo de transporte, interface),

beneficiou da adoção do Zustand. A simplicidade da API do Zustand reduziu drasticamente o boilerplate necessário para gerir estados aninhados, enquanto a sua integração natural com TypeScript assegurou type safety em todas as operações de estado. Para operações intensivas de mapa, onde o estado atualiza frequentemente em resposta as interações do utilizador ou dados recebidos de APIs, o Zustand demonstrou performance superior através do seu mecanismo otimizado de updates seletivos. Esta escolha permitiu ainda uma organização modular do estado em stores especializadas (rota, mapa, pesquisa), facilitando a manutenção e a testabilidade de cada domínio de forma independente.

3.3.4 Containerização com Docker

A containerização com Docker foi selecionada para garantir reproducibilidade total do ambiente de execução entre desenvolvimento, teste e produção. O isolamento de dependências fornecido pelos containers previne conflitos entre versões de bibliotecas e assegura que a aplicação se comporta de forma consistente independentemente do sistema hospedeiro. Esta abordagem simplificou significativamente o onboarding de novos membros da equipa, que puderam iniciar o desenvolvimento com um simples comando `docker-compose up`. A integração com Portainer adicionou uma camada de gestão visual que facilita operações de deploy, monitorização e troubleshooting em ambiente de produção, reduzindo a complexidade operacional e o potencial para erros de configuração manual.

4. Metodologia de Desenvolvimento

4.1 Metodologia SCRUM

A metodologia SCRUM foi adotada como base para a organização e execução do desenvolvimento do projeto, tendo sido adaptada ao contexto académico e à dimensão da equipa envolvida. Esta metodologia permite uma evolução contínua do sistema e uma resposta eficaz a alterações de requisitos ou decisões técnicas ao longo do tempo.

As cerimónias do SCRUM, nomeadamente o planeamento de sprint, a revisão e a retrospectiva, foram realizadas de forma simplificada, adequadas ao contexto de trabalho académico, mantendo o foco na reflexão contínua sobre o progresso e na melhoria do processo de desenvolvimento.

4.2 Planeamento do Projeto

O planeamento inicial do projeto teve como ponto de partida a análise das diferentes opções de tema propostas pela unidade curricular. Após a avaliação das alternativas disponíveis, foi selecionada a opção referente ao desenvolvimento de um sistema interativo de rotas e exploração de locais baseado em dados do OpenStreetMap.

A visão inicial do projeto delineava a criação de uma aplicação web responsiva, onde o utilizador pudesse definir uma origem e um destino, escolher um modo de transporte, e visualizar num mapa a rota calculada, juntamente com informações como distância e tempo estimado. Foram identificados requisitos funcionais obrigatórios, como a visualização do mapa, a pesquisa de locais, o cálculo de rotas e a exibição de marcadores. Em paralelo, definiram-se requisitos opcionais ou de extensão, que incluíam funcionalidades como a gestão de paragens intermédias (waypoints), a exibição de pontos de interesse próximos, ou a integração com um chatbot assistente. Estes últimos serviriam como margem para evolução e demonstração de capacidades avançadas, caso o desenvolvimento do núcleo progredisse conforme o planeado.

O planeamento temporal do projeto foi dinâmico e adaptou-se a uma mudança de direção fundamental a meio do desenvolvimento. No total, o trabalho foi organizado em quatro sprints, com durações e focos distintos.

O Sprint 1 foi concluído com sucesso. Seguindo a conceção inicial baseada no diagrama UML em Java, este sprint focou-se no desenvolvimento de uma aplicação desktop completa em Java Swing. As tarefas incluíram a implementação da interface gráfica com os componentes MainWindow e MapPanel, a criação das classes de modelo Location e Route, a integração das APIs OSRM e Nominatim através dos respetivos serviços, e a implementação da lógica de interação do utilizador (pesquisa, seleção de pontos no mapa, cálculo

e visualização de rotas). Ao final deste sprint, foi apresentado um produto funcional que cumpria os requisitos básicos do tema.

Após a discussão intercalar e a decisão de mudar para uma stack web, o planejamento subsequente foi totalmente reestruturado em três sprints curtos e intensivos, para acometer a reimplementação completa.

O Sprint 2 foi o sprint de migração arquitetural. O objetivo era estabelecer as novas fundações tecnológicas. A equipa concentrou-se em configurar o ambiente de desenvolvimento React com Vite e TypeScript, em estruturar o projeto em componentes, hooks, stores e serviços, e em integrar as bibliotecas centrais, sendo estas o Leaflet para mapas, o Zustand para estado e o Tailwind CSS para estilização. O entregável foi uma aplicação React básica capaz de exibir um mapa interativo, servindo de base sólida para o desenvolvimento rápido das funcionalidades.

O Sprint 3 foi dedicado à reconstrução das funcionalidades centrais. Com a nova arquitetura em funcionamento, foi reimplementado as funcionalidades básicas da versão Java no novo paradigma. Isto incluiu a criação dos serviços de API em TypeScript (`nominatim.service.ts`, `google-maps.service.ts`), o desenvolvimento dos componentes `LocationSearch` e `RouteLayer`, e a integração da lógica de cálculo e visualização de rotas. Ao final desta semana, a aplicação web já replicava a funcionalidade principal da versão Java, ou seja, pesquisa e cálculo de rotas.

O Sprint 4 focou-se no enriquecimento, polimento e funcionalidades avançadas, ou seja, a implementação completa do sistema de waypoints (paragens intermédias), o desenvolvimento do `ChatWidget` integrado com o workflow `n8n`, a otimização da interface para dispositivos móveis (responsividade), e o refino geral da experiência do utilizador. Assim, este sprint resultou na versão final e consolidada da aplicação `Map Route Explorer`, tendo esta sido mostrada na discussão final.

4.3 User Stories e Estimativas

A transformação dos requisitos funcionais em trabalho concreto foi realizada através da criação de User Stories (US), organizadas em sete épicos que cobriam desde as funcionalidades centrais até à infraestrutura do projeto.

O Épico 1 (Visualização e Navegação de Mapas) englobou as histórias fundamentais para a experiência base. A US-001 (Visualização do Mapa Base) permitia ao utilizador explorar geograficamente uma área através de um mapa interativo, envolvendo tarefas como configurar o React Leaflet, implementar o componente `MapContainer` e configurar os tiles do `OpenStreetMap`. A US-002 (Geolocalização do Utilizador) dava ao utilizador a capacidade de usar a sua

localização atual como ponto de partida, exigindo a implementação de um hook personalizado e a integração com a API do navegador. A US-003 (Auto-fit do Mapa) assegurava que, quando uma rota fosse calculada, o zoom se ajustaria automaticamente para mostrar todos os pontos relevantes.

O Épico 2 (Pesquisa e Geocodificação) focou-se na descoberta de locais. A US-004 (Pesquisa de Localização por Texto) era central, permitindo aos utilizadores encontrar locais escrevendo endereços, o que implicou a criação do serviço Google Places API e do componente LocationSearch com funcionalidades de autocomplete. A US-005 (Marcadores no Mapa) procura fornecer uma identificação visual clara dos pontos da rota, exigindo a criação de ícones personalizados e a sua renderização no mapa.

Para o Épico 3 (Cálculo e Gestão de Rotas), foram definidas histórias que constituíam o cerne da aplicação. A US-006 (Cálculo de Rota Básico) estabelecia a capacidade de calcular um percurso entre dois pontos, tarefa que envolveu a integração do serviço de routing e a criação do componente para desenhar a rota. A US-007 (Informações da Rota) garantia que o utilizador teria acesso a detalhes como distância e tempo estimado. A US-008 (Múltiplos Modos de Transporte) enriqueceu a aplicação ao permitir a escolha entre deslocação de carro, bicicleta ou a pé. A US-009 (Waypoints) adicionou complexidade e utilidade ao permitir a inclusão de paragens intermédias na rota, exigindo uma gestão de estado mais elaborada. As US-010 e US-011, embora planeadas para modos de transporte público e comparação de tempos, representavam funcionalidades avançadas de roadmap.

O Épico 4 (Pontos de Interesse) foi coberto pela US-012, que permitia aos utilizadores visualizar restaurantes, postos de combustível e outros pontos de interesse ao longo do seu percurso, integrando a Overpass API para obtenção de dados.

No Épico 5 (Integrações Externas), a US-013 facilitava a abertura da rota calculada diretamente na aplicação Google Maps para navegação turn-by-turn, enquanto a US-014 introduzia um chatbot assistente integrado com a plataforma n8n, permitindo uma interação mais natural para definir rotas.

O Épico 6 (UI/UX e Responsividade) assegurou a qualidade da experiência. A US-015 (Design Responsivo Mobile) foi crucial para garantir que a aplicação fosse utilizável em telemóveis, envolvendo a adaptação de layouts e controlos para touch. A US-016 (Tema Visual) e Acessibilidade focou-se na consistência visual e no cumprimento de diretrizes de contraste e navegação.

Por fim, o Épico 7 (Infraestrutura e DevOps) garantiu a robustez do processo de desenvolvimento. A US-017 (Deploy com Docker) permitia a containerização da aplicação para consistência entre ambientes, e a US-018 (CI/CD Pipeline) automatizava a verificação de qualidade de código em cada commit.

Esta versão inicial, correspondente ao Sprint 1 concluído, era uma aplicação totalmente operacional. A arquitetura, conforme documentada no diagrama UML fornecido, materializava-se em código numa interface gráfica construída com componentes Swing (MainWindow, MapPanel), uma camada de modelo com as classes Location e Route, e serviços como OSRMService e NominatimService que integravam as APIs externas. A aplicação permitia aos utilizadores visualizar um mapa interativo (com tiles do OpenStreetMap renderizados nativamente via Graphics2D), pesquisar locais através da API Nominatim, marcar pontos de origem e destino com cliques no mapa, e calcular rotas entre eles utilizando a API OSRM. O projeto Java estava, portanto, num estado avançado de desenvolvimento, com as funcionalidades nucleares já implementadas e testadas.

No entanto, durante a discussão intercalar de acompanhamento, o docente esclareceu que o uso de Java não era um requisito, sendo possível utilizar ou fazer o projeto da melhor forma considerada. Assim, e após uma análise, tornou-se evidente que a abordagem em Java Swing, embora funcional, apresentava limitações.

Esta reflexão levou a mudar completamente a stack tecnológica, abandonando a implementação em Java Swing em favor de uma arquitetura web moderna baseada em React.

Assim, esta mudança tecnológica exigiu um replaneamento completo do restante projeto, sendo um processo de evolução de uma aplicação Java desktop completa para uma aplicação web. O ajuste ao planeamento, com sprints mais curtos e intensivos, revelou a flexibilidade da metodologia SCRUM e fez com que fosse possível entregar um melhor produto em termos do esperado a nível do projeto.

5. Implementação Técnica

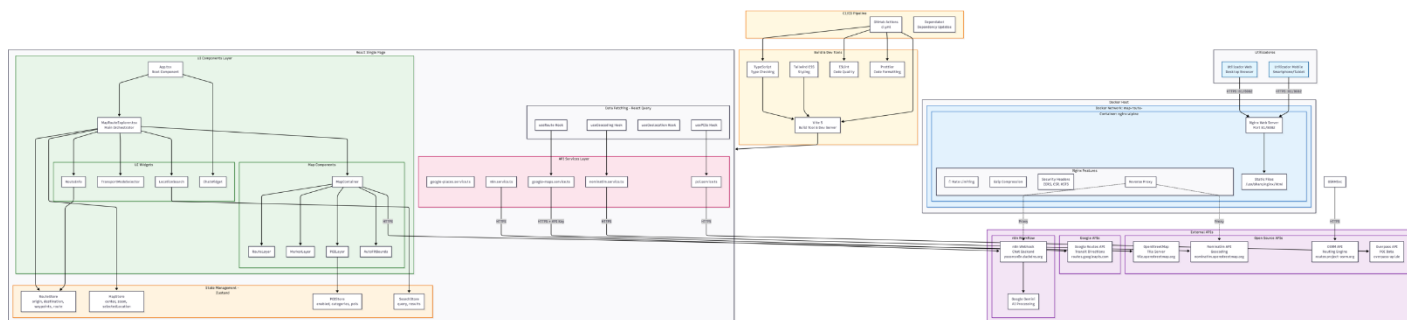


Figura 10 - Diagrama da Arquitetura completa do Map Route Explorer

A arquitetura técnica do sistema organiza-se em quatro áreas funcionais principais, claramente identificáveis na estrutura do diagrama apresentado. A primeira área engloba os Componentes de Interface, que correspondem aos elementos visuais com os quais o utilizador interage diretamente no ecrã, incluindo os componentes de mapa e os widgets de interação. A segunda área é dedicada à Gestão de Estado, implementada através de stores especializadas que mantêm e coordenam de forma consistente o estado interno da aplicação. A terceira área consiste na Obtenção de Dados, gerida por hooks baseados no React Query que orquestram a comunicação assíncrona com fontes externas, assegurando mecanismos eficientes de carregamento, sincronização e caching. A quarta e última área abrange os Serviços e APIs, constituída por módulos que abstraem e uniformizam a comunicação com os diversos serviços externos dos quais o sistema depende.

Este capítulo foca-se especificamente nestas quatro áreas funcionais, que representam o núcleo lógico e aplicacional do sistema, sendo responsáveis pelo comportamento observável da aplicação no navegador do utilizador. As restantes componentes visíveis no diagrama, nomeadamente as relacionadas com infraestrutura, segurança, integração contínua, deployment e serviços externos de suporte, serão abordados em capítulos posteriores do relatório. As secções seguintes deste capítulo, da 5.1 à 5.6, exploram em detalhe cada uma das áreas funcionais aqui introduzidas, mantendo como referência a visão global apresentada na Figura 10.

5.1 Módulo de Mapas

5.1.1 Visão Geral do Módulo de Mapas

O módulo de mapas constitui o núcleo visual e interativo da aplicação Map Route Explorer, sendo responsável por toda a renderização cartográfica, processamento de interações geográficas e apresentação visual das rotas calculadas. Esta componente fundamental da aplicação implementa uma arquitetura em camadas que combina robustez técnica com experiência de

utilizador fluida, garantindo que os elementos cartográficos são apresentados de forma clara, responsiva e interativa em todas as condições de utilização.

5.1.2 Componente MapContainer e Configuração Base

No centro deste módulo encontra-se o componente MapContainer, que serve como orquestrador principal de toda a funcionalidade de mapas. Este foi desenvolvido como um componente React que encapsula a biblioteca Leaflet através do wrapper React-Leaflet. Assim, o MapContainer configura o ambiente cartográfico base com parâmetros cuidadosamente ajustados. A configuração inicial define o centro do mapa nas coordenadas de Lisboa por defeito, com um nível de zoom que equilibra detalhe geográfico com contexto regional.

Os limites técnicos são rigorosamente controlados com o zoom mínimo de 3 previne visualizações excessivamente afastadas que perderiam utilidade prática, enquanto o zoom máximo de 19 oferece detalhe suficiente para navegação urbana precisa. Um aspeto crítico desta configuração é a desativação da funcionalidade worldCopyJump, que previne que o utilizador possa arrastar o mapa para cópias repetidas do mundo virtual, uma fonte comum de confusão em aplicações cartográficas.

5.1.3 Gestão de Limites Geográficos (BoundsEnforcer)

A gestão dos limites geográficos vai além da simples configuração inicial através do componente BoundsEnforcer, que implementa um sistema proativo de validação de coordenadas. Esta componente monitoriza continuamente a posição do mapa durante operações de arrasto, normalizando automaticamente as longitudes para o intervalo padrão de -180 a 180 graus e aplicando limites de latitude entre -85 e 85 graus.

A implementação utiliza event listeners do Leaflet que são cuidadosamente registados no momento de montagem do componente e removidos na desmontagem, prevenindo memory leaks em sessões prolongadas de utilização. Quando o sistema deteta que o utilizador tentou arrastar o mapa para fora dos limites válidos, reposiciona suavemente a vista para a posição normalizada mais próxima, mantendo a continuidade visual sem interrupções bruscas.

5.1.4 Gestão de Eventos e Interações no Mapa

A interação do utilizador com o mapa é gerida pelo componente MapEvents, que implementa um sistema complexo de processamento de eventos baseado no contexto operacional atual. Quando um utilizador clica no mapa, as coordenadas do clique são primeiro normalizadas para garantir consistência matemática.

A ação resultante do clique é determinada dinamicamente com base no estado waitingForInput, que pode indicar que o sistema está à espera de que o utilizador defina uma origem, um destino, ou um ponto intermédio. Esta lógica contextual permite fluxos de trabalho intuitivos, como a definição de pontos diretamente

através de cliques no mapa, sem necessidade de introdução manual de coordenadas.

5.1.5 Otimização de Atualizações e Performance

A atualização da vista do mapa durante operações de navegação é otimizada para performance através de um sistema de debouncing implícito. Quando o utilizador arrasta ou faz zoom no mapa, o componente MapEvents verifica mudanças de 0.0001 graus para centro antes de atualizar o estado global da aplicação.

Este limiar previne atualizações de estado desnecessárias durante micro-ajustes do mapa, reduzindo significativamente o número de re-renderizações dos componentes React. O sistema utiliza uma referência mutable para prevenir loops infinitos causados por atualizações recursivas de estado.

5.1.6 Camadas de Visualização Cartográfica

Sobre esta base cartográfica robusta, múltiplas camadas de renderização especializadas trabalham em conjunto para apresentar informações geográficas de forma clara e organizada.

1. RouteLayer

A RouteLayer é responsável pela visualização das rotas calculadas, convertendo diferentes formatos de geometria (polylines da Google Maps e GeoJSON do OSRM) numa representação visual unificada. Os estilos visuais são diferenciados por modo de transporte, permitindo identificação imediata do tipo de percurso.

2. MarkerLayer

A MarkerLayer gere a exibição de marcadores visuais para origem, destino e pontos intermédios. A utilização de divlcon permite marcadores HTML personalizados, escaláveis e facilmente estilizados, cada um com popups informativos contextuais.

3. POILayer

A POILayer introduz contexto adicional ao mapa ao exibir pontos de interesse relevantes. A renderização é condicional e limitada para evitar sobrecarga visual, com integração otimizada com a Overpass API.

5.1.7 Ajuste Automático da Vista (AutoFitBounds)

O componente AutoFitBounds ajusta automaticamente a vista do mapa para incluir todos os pontos relevantes sempre que uma rota é calculada ou modificada. O sistema calcula o bounding box mínimo que engloba origem, destino e waypoints, aplicando padding e limites de zoom cuidadosamente calibrados para garantir clareza visual e conforto na navegação.

5.2 Sistema de Rotas

5.2.1 Visão Geral do Sistema de Rotas

O sistema de cálculo de rotas constitui o núcleo funcional e algorítmico da aplicação Map Route Explorer, transformando pares de coordenadas geográficas em percursos navegáveis completos com estimativas precisas de tempo, distância e geometria espacial.

5.2.2 Estratégia de Provedores de Routing

O motor principal é a Google Maps Directions API, selecionada pela sua precisão, cobertura global e funcionalidades avançadas. Como alternativa de fallback, o sistema integra o OSRM, embora atualmente o OSRM não seja utilizado automaticamente no fluxo principal, este está disponível como serviço mas não há lógica de fallback automático implementada.

5.2.3 Integração com a Google Maps Directions API

A implementação do serviço Google Maps Directions segue uma abordagem modular que encapsula toda a complexidade da interação com a API JavaScript da Google. O carregamento do SDK é realizado de forma assíncrona através da função `loadGoogleMapsAPI`, que implementa carregamento lazy e múltiplas salvaguardas contra duplicação.

Após o carregamento, é instanciado um objeto `DirectionsService` que serve como interface primária para os pedidos de cálculo de rotas.

5.2.4 Construção e Execução dos Pedidos de Rota

Cada pedido de rota é construído como um objeto `DirectionsRequest`, onde origem, destino, modo de transporte e parâmetros adicionais são cuidadosamente definidos. O sistema suporta configurações avançadas para o modo de condução, incluindo modelos de tráfego e hora de partida atual, garantindo estimativas realistas.

5.2.5 Processamento e Normalização da Resposta

Quando a resposta é recebida, o sistema inicia um processo de parsing que converte o formato proprietário da Google numa representação interna padronizada. A geometria da rota é reconstruída através da decodificação de `polylines`, enquanto métricas como distância e duração são agregadas a partir dos diferentes legs da rota.

As instruções de navegação textual são extraídas, limpas de marcação HTML e organizadas sequencialmente para possível utilização futura.

5.2.6 Integração com o OSRM como Provedor Secundário

O serviço OSRM implementa uma interface alternativa baseada em endpoints HTTP REST. A resposta GeoJSON é processada diretamente, com extração

simples da geometria e métricas. Dada a menor sofisticação do OSRM, o sistema aplica ajustes empíricos às estimativas de duração para aproximar os valores das condições reais.

5.2.7 Suporte a Diferentes Modos de Transporte

O sistema inclui lógica especializada para automóvel, bicicleta e pedestrianismo. Para automóvel, são utilizados dados de tráfego em tempo real quando disponíveis. Para bicicleta e caminhada, são aplicados ajustes baseados em velocidades médias realistas e características típicas de cada modo de deslocação.

5.2.8 Tratamento de Erros e Resiliência

O tratamento de erros é implementado em múltiplas camadas, distinguindo problemas de rede, erros de autenticação, indisponibilidade temporária e erros específicos de domínio. Sempre que ocorre uma falha, o sistema preserva o último estado válido, evitando perda de contexto para o utilizador.

5.3 Funcionalidade de Waypoints

5.3.1 Visão Geral da Funcionalidade de Waypoints

A funcionalidade de waypoints estende o modelo clássico de planeamento de rotas ponto-a-ponto, permitindo a definição de percursos com múltiplas paragens intermédias. Esta capacidade é essencial para representar cenários reais de deslocação, como viagens com paragens planeadas, recolha de passageiros ou visitas sequenciais a locais de interesse.

5.3.2 Modelo de Waypoints e Restrições

Cada waypoint é representado internamente como uma instância do modelo Location, contendo coordenadas geográficas obrigatórias (latitude e longitude) e metadados opcionais, como nome descritivo e endereço. Os waypoints são mantidos numa estrutura ordenada, preservando explicitamente a sequência definida pelo utilizador, que corresponde à ordem de visita ao longo da rota.

Foi imposto um limite máximo de cinco waypoints por rota. Esta decisão resulta de uma combinação de fatores técnicos e de usabilidade. Do ponto de vista técnico, os serviços de routing utilizados apresentam limitações práticas no número de paragens intermédias suportadas de forma eficiente. Do ponto de vista da experiência do utilizador, um número elevado de paragens tende a tornar a interface visualmente confusa, especialmente em dispositivos móveis. O limite é aplicado de forma preventiva na interface, impedindo a adição de novos waypoints quando o máximo é atingido e fornecendo feedback imediato ao utilizador.

5.3.3 Interação do Utilizador

A definição de waypoints é integrada de forma progressiva no fluxo de planeamento da rota. Após a definição da origem, a interface passa a permitir a adição de paragens intermédias antes da definição do destino final. Cada waypoint pode ser introduzido através de diferentes mecanismos de interação, incluindo pesquisa textual por localização, utilização da posição atual do utilizador ou seleção direta no mapa.

A interface apresenta cada waypoint de forma individualizada, com identificação sequencial clara (Paragem 1, Paragem 2, etc.), reforçando visualmente a ordem do percurso. O utilizador pode remover qualquer waypoint intermédio, sendo a reindexação dos restantes efetuada automaticamente para manter consistência visual e lógica.

5.3.4 Processamento de Resultados e Validação

Quando uma rota com waypoints é calculada com sucesso, o sistema agrega corretamente os segmentos intermédios devolvidos pela API, produzindo métricas globais coerentes de distância e duração. Validações adicionais asseguram que os waypoints se encontram dentro de limites geográficos válidos e que não existem duplicações ou paragens excessivamente próximas que possam comprometer a qualidade da rota.

Sempre que uma configuração de waypoints resulta num erro de cálculo, o sistema fornece mensagens de erro contextualizadas, permitindo ao utilizador ajustar ou remover pontos intermédios problemáticos sem perder o restante planeamento.

5.4 Integração do Chatbot

5.4.1 Visão Geral do Chatbot

A integração do chatbot no Map Route Explorer introduz uma camada adicional de interação baseada em linguagem natural, com o objetivo de tornar o planeamento de rotas mais acessível, intuitivo e orientado ao utilizador. Esta funcionalidade permite que o utilizador formule pedidos em linguagem natural, como sugestões de rotas, esclarecimento de dúvidas ou pedidos de ajuda, complementando a interação gráfica tradicional baseada em formulários e cliques no mapa.

5.4.2 Arquitetura da Integração

O chatbot é implementado como um componente isolado da interface, designado ChatWidget, que encapsula toda a lógica de apresentação, envio e receção de mensagens. A lógica de processamento conversacional não é executada localmente na aplicação, sendo delegada a um workflow externo gerido pela

plataforma n8n. Esta decisão permite desacoplar completamente a interface do utilizador da lógica de interpretação de linguagem natural.

5.4.3 Fluxo de Comunicação

Quando o utilizador envia uma mensagem através do ChatWidget, o conteúdo é encapsulado num pedido HTTP e enviado para um endpoint webhook exposto pelo n8n. O workflow no n8n recebe a mensagem, processa-a utilizando o modelo de linguagem Gemini, e devolve uma resposta estruturada. Esta resposta pode conter apenas texto explicativo ou, em cenários mais avançados, instruções interpretáveis pela aplicação, como sugestões de locais ou parâmetros de rota.

5.4.4 Processamento de Respostas

As respostas do chatbot são apresentadas na interface de conversação de forma sequencial, mantendo o histórico da interação durante a sessão ativa. O sistema garante que a resposta do chatbot não altera diretamente o estado da aplicação sem validação explícita do utilizador, prevenindo comportamentos inesperados. Desta forma, o chatbot atua como um assistente consultivo e não como um agente autónomo com controlo direto sobre o planeamento da rota.

5.4.5 Gestão de Estado e Isolamento Funcional

O estado da conversação é mantido localmente no componente ChatWidget, separado do estado global da aplicação. Esta separação assegura que falhas, atrasos ou erros na comunicação com o chatbot não impactam negativamente o funcionamento do planeamento de rotas ou da visualização do mapa.

5.5 Pontos de Interesse

5.5.1 Enquadramento da Funcionalidade de POIs

A funcionalidade de pontos de interesse (POIs) acrescenta uma camada contextual ao Map Route Explorer, permitindo explorar informação relevante ao longo da rota ou na área atualmente visível no mapa.

Esta funcionalidade complementa o planeamento de rotas sem interferir diretamente com o seu cálculo.

5.5.2 Integração com a Overpass API

A obtenção de pontos de interesse baseia-se na integração com a Overpass API, que permite consultar dados do OpenStreetMap de forma flexível. As queries são formuladas para filtrar categorias relevantes e restringir os resultados à área de interesse.

5.5.3 Gestão e Cache de Dados

Os dados obtidos são geridos através de hooks especializados que encapsulam a lógica de comunicação com a API externa, incluindo mecanismos de cache e controlo de frequência de pedidos. Esta abordagem melhora a performance e reduz chamadas redundantes.

5.5.4 Visualização dos Pontos de Interesse

Os POIs são apresentados através de uma camada dedicada no mapa, com marcadores de dimensão reduzida e codificação cromática por categoria. Esta separação garante que a informação contextual não compromete a legibilidade da rota principal.

5.5.5 Limitação Visual e Usabilidade

Para evitar sobrecarga visual, o número de pontos apresentados é limitado com base na proximidade à rota ou ao centro da vista atual. Esta abordagem privilegia utilidade e clareza, especialmente em áreas urbanas densas.

5.6 Gestão de Estado

5.6.1 Visão Geral da Gestão de Estado

A gestão de estado da aplicação Map Route Explorer desempenha um papel central na coordenação entre os diversos módulos funcionais, garantindo consistência, previsibilidade e sincronização entre a interface do utilizador, o mapa interativo e os serviços externos. Dada a natureza altamente interativa da aplicação, com múltiplos componentes dependentes de dados partilhados, foi adotada uma abordagem de estado global baseada na biblioteca Zustand.

5.6.2 Estrutura das Stores Principais

A aplicação organiza o estado global em múltiplas stores especializadas, cada uma responsável por um domínio funcional específico. Esta separação segue o princípio de responsabilidade única e contribui para uma arquitetura modular e facilmente extensível.

A RouteStore gere toda a informação relacionada com o planeamento de rotas, incluindo origem, destino, pontos intermédios (waypoints), modo de transporte selecionado e resultados do cálculo de rota. Esta store funciona como fonte única de verdade para o sistema de routing.

A MapStore é responsável pelo estado da visualização cartográfica, incluindo centro do mapa, nível de zoom, limites visíveis e flags relacionadas com interações do utilizador. Esta separação permite que o mapa reaja a alterações de estado sem depender diretamente de componentes específicos.

A POIStore gere o estado relacionado com pontos de interesse, como lista de POIs carregados, filtros ativos e estados de carregamento.

A SearchStore é responsável pela gestão do estado associado à pesquisa de localizações. Esta store mantém a query introduzida pelo utilizador, os resultados devolvidos pelo serviço de geocodificação e o estado de carregamento da pesquisa.

5.6.3 Ações e Atualização de Estado

Cada store define explicitamente as ações permitidas para modificação do estado, como `setOrigin`, `setDestination`, `addWaypoint` ou `clearRoute`. Estas ações são implementadas de forma imutável, retornando sempre novos objetos de estado em vez de modificar diretamente os existentes.

5.6.4 Integração com Componentes React

O acesso ao estado é realizado através de hooks fornecidos pelo Zustand, permitindo que cada componente subscreva apenas as partes do estado de que necessita. Esta granularidade reduz re-renderizações desnecessárias e melhora significativamente a performance da aplicação, especialmente em cenários com múltiplas atualizações em rápida sucessão.

Componentes como `MapContainer`, `LocationSearch` e `RouteLayer` reagem automaticamente a alterações relevantes no estado global, mantendo sincronização contínua entre interface e lógica de negócio.

5.6.5 Sincronização entre Estado e APIs Externas

A gestão de estado desempenha também um papel fundamental na coordenação com serviços externos. Alterações no estado da rota, como definição de origem, destino ou waypoints, desencadeiam automaticamente chamadas aos serviços de routing através de hooks especializados. Os resultados destas chamadas são então refletidos novamente no estado global, fechando o ciclo de sincronização.

Este fluxo unidirecional de dados facilita o raciocínio sobre o comportamento da aplicação e simplifica a depuração de problemas.

6. Qualidade de Código e CI/CD

Este capítulo descreve as práticas adotadas no projeto Map Route Explorer no que diz respeito à garantia de qualidade de código e à automação de processos através de integração contínua e entrega contínua (CI/CD). O objetivo principal destas práticas é assegurar um código consistente, fiável, facilmente manutenível e alinhado com boas práticas modernas de engenharia de software, reduzindo erros em produção e aumentando a confiança no processo de desenvolvimento.

6.1 Pipeline CI/CD

O projeto integra um pipeline de Integração Contínua (CI) implementado através do GitHub Actions, acionado automaticamente em eventos de push e pull request para o repositório. Esta configuração garante que todas as alterações ao código são validadas de forma sistemática antes de serem integradas na base principal do projeto, promovendo uma cultura de qualidade e prevenindo a introdução de regressões.

O pipeline é executado num ambiente Linux (ubuntu-latest) e segue uma sequência de etapas bem definida. Inicialmente, o código-fonte é obtido diretamente do repositório através da ação de checkout. De seguida, é configurado o ambiente Node.js na versão 20, alinhada com a utilizada no Dockerfile de produção, assegurando consistência entre ambientes de desenvolvimento, CI e deployment. A instalação das dependências é realizada com o comando `npm ci`, privilegiando builds reprodutíveis e evitando ambiguidades na resolução de versões.

Após a instalação das dependências, o pipeline executa um conjunto de verificações automáticas destinadas a validar a qualidade do código. Estas incluem a análise estática com o ESLint, a verificação de formatação com o Prettier, a validação de tipos com o TypeScript e, por fim, o processo de build da aplicação. A execução sequencial destas etapas garante que apenas código que cumpre os critérios definidos de qualidade, consistência e correção é considerado válido. Qualquer falha interrompe imediatamente o pipeline, fornecendo feedback rápido aos desenvolvedores.

Embora o pipeline esteja focado na integração contínua, a sua integração com o processo de deployment baseado em Docker permite uma extensão natural para práticas de entrega contínua, uma vez que o código validado pode ser posteriormente empacotado e distribuído de forma automatizada.

6.2 Ferramentas de Qualidade de Código

A qualidade do código é reforçada através da utilização de várias ferramentas complementares, integradas tanto no fluxo de desenvolvimento local como no pipeline de CI. O projeto é desenvolvido integralmente em TypeScript, permitindo

a detecção precoce de erros através de tipagem estática e a definição explícita de contratos entre componentes, serviços e módulos. Esta abordagem melhora significativamente a robustez do sistema e facilita a sua evolução ao longo do tempo.

O ESLint é utilizado para análise estática do código, assegurando o cumprimento de regras de estilo, boas práticas e prevenção de padrões de código problemáticos. A presença de regras específicas para TypeScript contribui para uma maior consistência e para a redução de erros comuns associados ao uso de tipos dinâmicos. Em situações excepcionais, determinadas regras podem ser explicitamente desativadas de forma controlada, refletindo um uso consciente da ferramenta.

A formatação do código é assegurada pelo Prettier, que impõe um estilo uniforme em todo o projeto. A verificação automática da formatação durante o pipeline de CI garante que o código mantém uma aparência consistente, independentemente do programador ou do ambiente utilizado, reduzindo fricção em revisões de código.

Para validação adicional da correção do código, o projeto utiliza testes automatizados com o Vitest, uma framework de testes integrada no ecossistema Vite. A arquitetura modular do sistema, com separação clara entre componentes, hooks e serviços, facilita a criação de testes unitários e contribui para uma maior confiabilidade do código. Embora o foco principal esteja na validação estrutural e funcional, a inclusão desta ferramenta reforça a estratégia global de qualidade.

6.3 Badge de Integração Contínua

Como complemento ao pipeline de CI, o projeto pode expor um *badge* de integração contínua no repositório, refletindo o estado atual das verificações automáticas. Este badge fornece uma indicação visual imediata sobre o sucesso ou falha do pipeline, aumentando a transparência do processo de desenvolvimento.

O estado esperado do badge é representado como ativo e funcional, indicando que o código presente no ramo principal passou com sucesso por todas as etapas de validação definidas. Esta prática contribui para reforçar a confiança na estabilidade do projeto e incentiva a manutenção contínua de elevados padrões de qualidade.

6.4 Dependabot

A gestão de dependências é suportada pelo Dependabot, configurado para monitorizar automaticamente as dependências do ecossistema npm utilizadas no projeto. O Dependabot verifica semanalmente a existência de atualizações e vulnerabilidades conhecidas, criando pull requests automáticos sempre que são detetadas novas versões relevantes, dentro de limites previamente definidos.

A utilização do Dependabot apresenta benefícios significativos, nomeadamente a redução de riscos de segurança associados a dependências desatualizadas, a diminuição do esforço manual na manutenção do projeto e a promoção de uma atualização contínua e controlada das bibliotecas utilizadas. Integrado com o pipeline de CI, cada atualização proposta é automaticamente validada, assegurando que as alterações não introduzem regressões ou incompatibilidades.

7. Deployment e DevOps

Este capítulo descreve a estratégia de deployment e as práticas de DevOps adotadas no projeto Map Route Explorer, apresentando de forma integrada a arquitetura de deployment, a utilização de containers Docker, o processo de deploy através do Portainer e os mecanismos de monitorização e registo de logs. As decisões técnicas descritas ao longo deste capítulo têm como objetivo garantir consistência entre ambientes, facilidade de manutenção, desempenho adequado em produção e observabilidade do sistema.

7.1 Arquitetura de Deployment

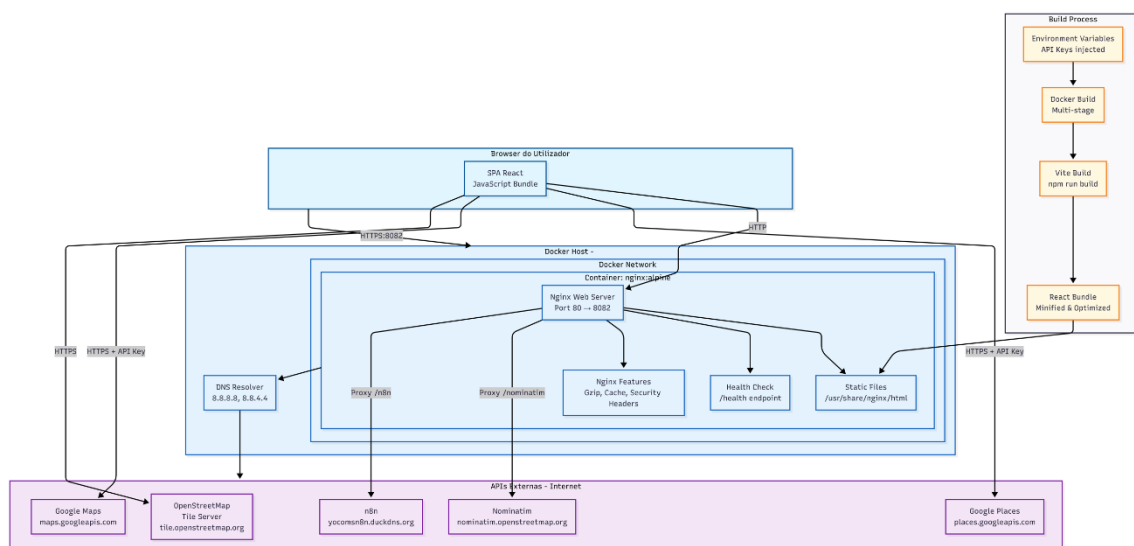


Figura 11 - Diagrama de Deployment

A arquitetura de deployment do sistema foi concebida com base numa separação clara de ambientes, permitindo distinguir o contexto de desenvolvimento do ambiente de produção. Em desenvolvimento, a aplicação é executada localmente recorrendo ao servidor de desenvolvimento do Vite, o que possibilita hot-reload, iteração rápida e acesso direto às APIs externas durante a fase de implementação e testes. Este ambiente privilegia a produtividade do programador e a deteção precoce de erros.

Em produção, a aplicação é distribuída sob a forma de um container Docker, onde o código React é previamente compilado e servido como uma aplicação estática através do Nginx. As variáveis de configuração relevantes, como URLs de serviços externos e chaves de API, são injetadas no momento de build, assegurando que o comportamento da aplicação em produção é previsível e controlado. Esta abordagem reduz discrepâncias entre ambientes e contribui para a estabilidade do sistema.

Do ponto de vista arquitetural, o utilizador acede à aplicação através de um browser web, sendo os pedidos HTTP encaminhados para um container Docker que integra o servidor Nginx. Este servidor é responsável tanto por servir os ficheiros estáticos da aplicação React como por atuar como proxy reverso para determinados serviços externos, nomeadamente a API de geocodificação Nominatim e o webhook do n8n utilizado pelo chatbot. Outras APIs, como o serviço de routing OSRM ou a Google Maps API, são consumidas diretamente pelo frontend via HTTPS. Esta arquitetura simplifica a infraestrutura, minimiza problemas de CORS e centraliza políticas de segurança e controlo de acesso.

A Figura 11 ilustra de forma integrada a arquitetura de deployment do sistema, evidenciando o processo de build, a estrutura do container Docker, o papel do servidor Nginx e os fluxos de comunicação entre o browser do utilizador e as APIs externas. Esta visão serve de base para as secções seguintes, que detalham os aspetos de containerização, deployment e monitorização.

7.2 Containerização com Docker

A containerização do sistema é realizada através do Docker, recorrendo a um Dockerfile multi-stage que separa claramente as fases de build e de execução. Na primeira fase, baseada na imagem `node:20-alpine`, são instaladas as dependências do projeto utilizando `npm ci`, garantindo builds reproduzíveis. Nesta fase são também definidas as variáveis de ambiente necessárias ao Vite, permitindo que estas sejam incorporadas no bundle final da aplicação. De seguida, é executado o processo de build, originando os artefactos estáticos da aplicação React.

Na segunda fase do Dockerfile é utilizada a imagem `nginx:alpine`, para a qual são copiados exclusivamente os ficheiros gerados no build. Esta fase inclui ainda uma configuração personalizada do Nginx, adaptada ao funcionamento de uma Single Page Application, e a definição de um health check interno. A utilização de um Dockerfile multi-stage resulta numa imagem final mais leve, segura e adequada a ambientes de produção.

A orquestração do container é assegurada através do ficheiro `docker-compose.yml`, que define o processo de build, a exposição da aplicação na porta 8082, a associação a uma rede Docker dedicada e uma política de reinício automático. O Docker Compose inclui igualmente a definição de um health check

baseado num endpoint HTTP, permitindo verificar de forma contínua o estado da aplicação. Esta abordagem simplifica o deployment e facilita a replicação do ambiente em diferentes servidores.

O Nginx assume um papel central na estratégia de deployment. Para além de servir a aplicação React como SPA, utilizando a diretiva `try_files` para encaminhar todas as rotas para o ficheiro `index.html`, o servidor encontra-se configurado para aplicar compressão `gzip`, cache agressivo para ficheiros estáticos e headers de segurança. Adicionalmente, o Nginx funciona como proxy reverso para serviços externos, evitando problemas de CORS no frontend e permitindo um maior controlo sobre timeouts, headers e políticas de acesso.

7.3 Deploy via Portainer

O deployment da aplicação pode ser realizado através do Portainer, como foi no nosso caso, uma ferramenta de gestão visual de ambientes Docker que simplifica significativamente as operações de deploy. O processo consistiu na configuração de uma stack baseada no ficheiro `docker-compose.yml`, na definição das variáveis de ambiente necessárias através de um ficheiro `.env` e na execução do build e arranque dos containers diretamente a partir da interface gráfica.

A utilização do Portainer apresenta várias vantagens, nomeadamente a redução da complexidade operacional, a facilidade de monitorização do estado dos containers e a possibilidade de realizar redeloys de forma rápida e controlada. Esta abordagem é particularmente adequada para contextos académicos ou projetos de pequena e média escala, onde é desejável um equilíbrio entre controlo técnico e simplicidade de gestão.

7.4 Monitorização e Logs

A monitorização do sistema é suportada por mecanismos simples mas eficazes. A aplicação disponibiliza um endpoint `/health`, utilizado tanto pelo Docker como por ferramentas externas para verificar a disponibilidade do serviço. Este health check permite a deteção automática de falhas e o reinício do container sempre que necessário, contribuindo para a resiliência da aplicação.

Relativamente aos logs, o Nginx encontra-se configurado para registar logs de acesso e de erro, possibilitando a análise de tráfego e o diagnóstico de problemas. Estes logs podem ser consultados diretamente no sistema de ficheiros do container ou através das ferramentas disponibilizadas pelo Docker e pelo Portainer. Em conjunto, estes mecanismos fornecem visibilidade suficiente sobre o comportamento da aplicação em produção e suportam atividades de manutenção e troubleshooting.

8. Segurança

A segurança do sistema Map Route Explorer foi considerada de forma transversal ao longo de todo o processo de conceção e implementação, tendo em conta o seu contexto de aplicação web, o consumo intensivo de APIs externas e a inexistência de autenticação de utilizadores. Este capítulo descreve o modelo de ameaças adotado, os principais controlos de segurança implementados, a gestão de credenciais sensíveis, a utilização de headers de segurança e as considerações relacionadas com privacidade, integrando e justificando as decisões técnicas tomadas.

8.1 Modelo STRIDE

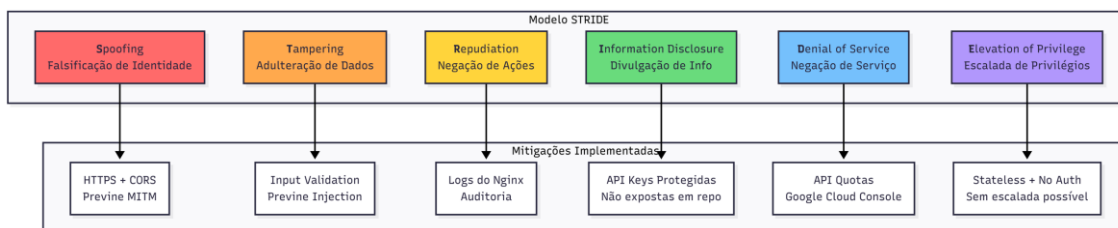


Figura 12 - Matriz de Ameaças STRIDE

A análise de ameaças do sistema baseia-se no modelo STRIDE, amplamente utilizado para a identificação sistemática de riscos de segurança em sistemas distribuídos. Este modelo permitiu classificar as ameaças potenciais em seis categorias principais, sendo estas, Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service e Elevation of Privilege. A aplicação do modelo STRIDE ao Map Route Explorer possibilitou uma avaliação estruturada da superfície de ataque do sistema e a definição de medidas de mitigação proporcionais ao seu contexto e complexidade.

Conforme ilustrado no diagrama STRIDE apresentado na Figura 12, a ameaça de Spoofing está associada à possibilidade de um atacante tentar imitar um cliente legítimo ou interceptar comunicações entre o utilizador e o sistema. Este risco é mitigado através do uso exclusivo de comunicações HTTPS, complementadas por políticas de CORS adequadamente configuradas, garantindo a confidencialidade e a integridade dos dados transmitidos e prevenindo ataques do tipo man-in-the-middle.

A categoria Tampering, relacionada com a adulteração de dados em pedidos enviados pelo utilizador ou encaminhados para serviços externos, é mitigada principalmente através de mecanismos de validação de input no frontend, assegurando que apenas dados bem formados e dentro de limites esperados são processados pela aplicação.

No que respeita à Repudiation, embora o sistema não implemente autenticação nem ações persistentes associadas a utilizadores, são mantidos registos de

acesso e erro no servidor Nginx, permitindo um nível básico de auditoria e análise de eventos em caso de comportamento anômalo ou falhas operacionais.

A ameaça de Information Disclosure assume particular relevância numa aplicação frontend, onde o código é executado no browser do utilizador. Para mitigar este risco, as chaves de acesso a APIs externas são geridas de forma segura, não sendo expostas em repositórios públicos nem incluídas diretamente no código-fonte versionado.

A categoria Denial of Service é considerada uma das mais relevantes, dada a natureza pública da aplicação e a dependência de serviços externos. Este risco é mitigado através da utilização de quotas definidas ao nível das APIs externas, nomeadamente no Google Cloud Console, prevenindo abusos e garantindo a disponibilidade do serviço.

Por fim, a Elevação de Privilégios apresenta impacto reduzido, uma vez que o sistema é essencialmente stateless, não implementa autenticação, nem define níveis de permissão diferenciados. A ausência de operações privilegiadas e de estado persistente impede a exploração de mecanismos de escalada, conforme sintetizado no diagrama de mitigação associado ao modelo STRIDE.

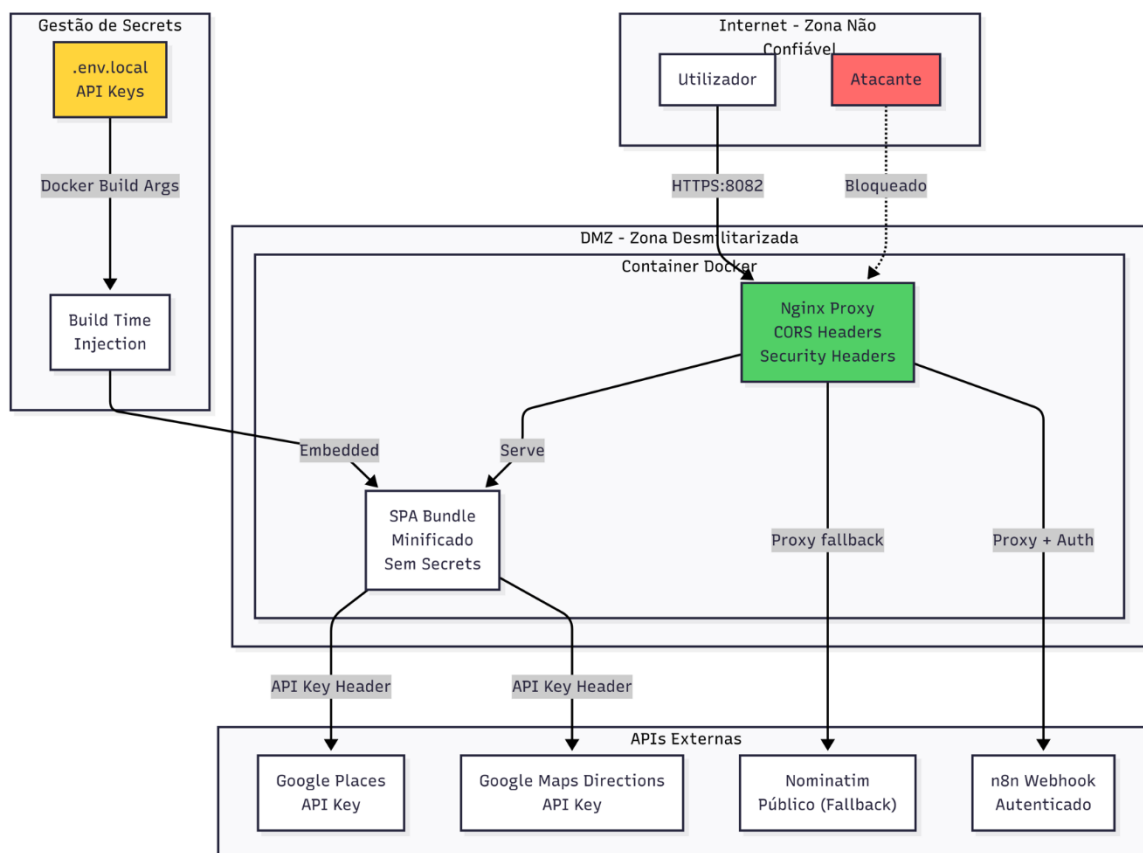


Figura 13 - Diagrama de Segurança (Fluxo de Dados)

Os controlos de segurança implementados no sistema distribuem-se por várias camadas, refletindo uma abordagem de defesa em profundidade, adequada à arquitetura frontend-centrada do Map Route Explorer. Conforme representado no diagrama de camadas de segurança (Figura 13), a arquitetura encontra-se organizada em zonas com diferentes níveis de confiança, distinguindo claramente a Internet, a DMZ e os serviços externos.

O acesso à aplicação é efetuado a partir da Internet, onde se encontram tanto utilizadores legítimos como potenciais atacantes. Todo o tráfego de entrada é encaminhado para a DMZ, onde reside o container Docker com o servidor Nginx, que atua como principal fronteira de segurança do sistema. O Nginx encontra-se configurado para servir a aplicação na porta 80, com HTTPS sendo tratado por um proxy reverso externo.

A aplicação do cliente é distribuída sob a forma de um bundle SPA minificado, servido diretamente pelo Nginx, não contendo qualquer informação sensível ou secrets embebidos em runtime. A gestão de chaves de API é realizada de forma controlada através de ficheiros de ambiente (.env), sendo estas injetadas apenas em tempo de build, reduzindo o risco de exposição acidental em repositórios ou logs.

Relativamente à comunicação com serviços externos, o sistema adota uma abordagem seletiva. As chamadas à Google Places API e à Google Maps Directions API são realizadas diretamente pelo frontend via HTTPS, utilizando chaves de API protegidas por restrições de domínio. Por outro lado, serviços como a Nominatim API e o webhook do n8n são acedidos através do proxy Nginx, permitindo um maior controlo sobre headers, autenticação e políticas de acesso.

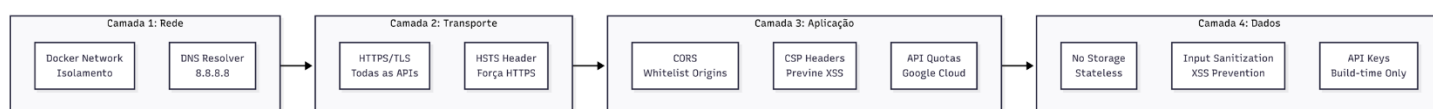


Figura 14 - Controlos de Segurança Implementados

A gestão de CORS é realizada centralmente no Nginx, que atua como proxy reverso para determinados serviços externos, como a API de geocodificação Nominatim e o webhook do n8n. Esta configuração permite definir explicitamente uma whitelist de origens autorizadas, bem como os métodos HTTP e headers permitidos, reduzindo o risco de utilização abusiva da aplicação a partir de origens não confiáveis.

Conforme ilustrado no diagrama das camadas de segurança (Figura 14), esta medida insere-se na Camada de Aplicação, complementando controlos existentes noutras camadas. Ao nível da Camada de Rede, o isolamento proporcionado pela Docker Network limita a exposição dos serviços internos. Na

Camada de Transporte, a utilização consistente de HTTPS/TLS e a aplicação do header HSTS asseguram a confidencialidade e integridade das comunicações. Finalmente, na Camada de Dados, a natureza stateless da aplicação, aliada à sanitização de inputs e à gestão de chaves de API exclusivamente em tempo de build, reduz o risco de fuga ou persistência indevida de informação sensível.

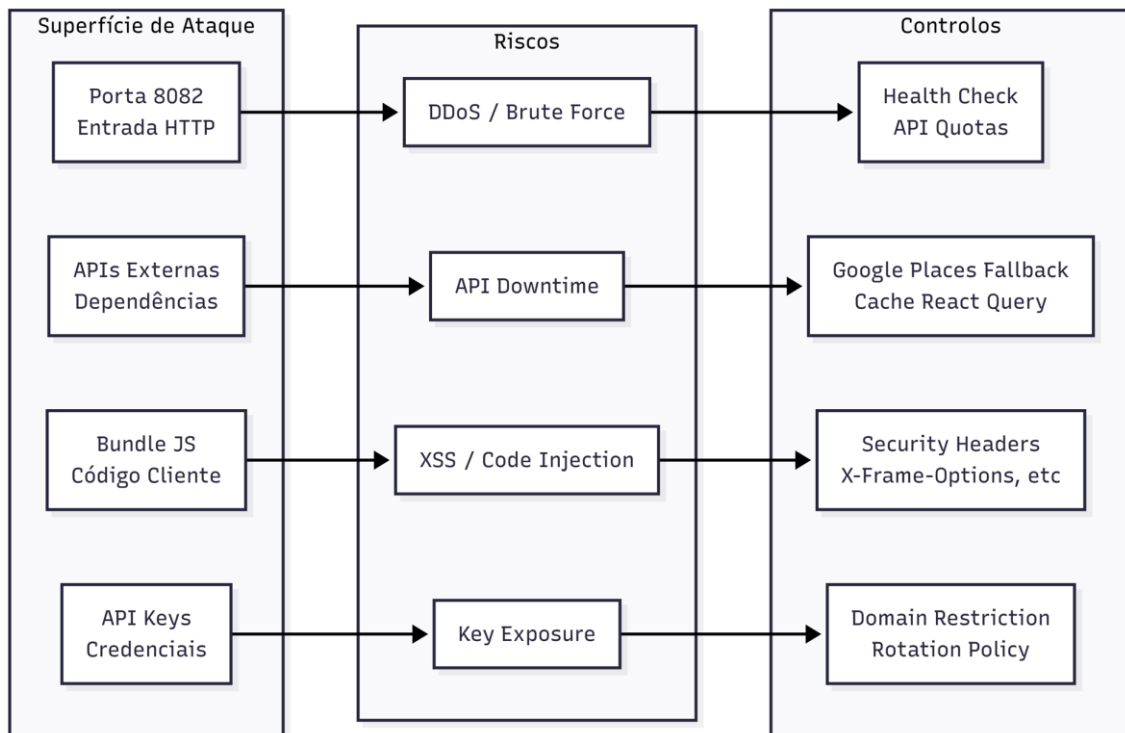


Figura 15 - Superfície de Ataque e os respetivos controlos

A superfície de ataque do sistema concentra-se em quatro pontos principais: a porta de entrada HTTP na porta 8082, as dependências de APIs externas, o bundle JavaScript executado no cliente e a gestão de chaves de acesso a serviços terceiros. Cada um destes pontos introduz riscos específicos, respetivamente associados a ataques de negação de serviço, indisponibilidade de serviços externos, vulnerabilidades do tipo XSS ou injeção de código, e exposição de credenciais.

Para mitigar estes riscos, foram aplicados controlos direcionados a cada vetor de ataque. A entrada HTTP é protegida através de mecanismos de health check e controlo indireto de carga via quotas das APIs externas. A dependência de serviços externos é mitigada por estratégias de cache com React Query e mecanismos de fallback. A exposição do código cliente é protegida pela aplicação de security headers, como X-Frame-Options e políticas de isolamento de conteúdo. Por fim, a gestão de chaves de API é reforçada através de restrições por domínio e políticas de rotação periódica.

A correspondência entre pontos de entrada, riscos identificados e respetivos controlos encontra-se resumida no diagrama de superfície de ataque e mitigação (Figura 15), permitindo uma visualização clara das ameaças consideradas e das medidas de proteção implementadas.

8.3 Gestão de API Keys

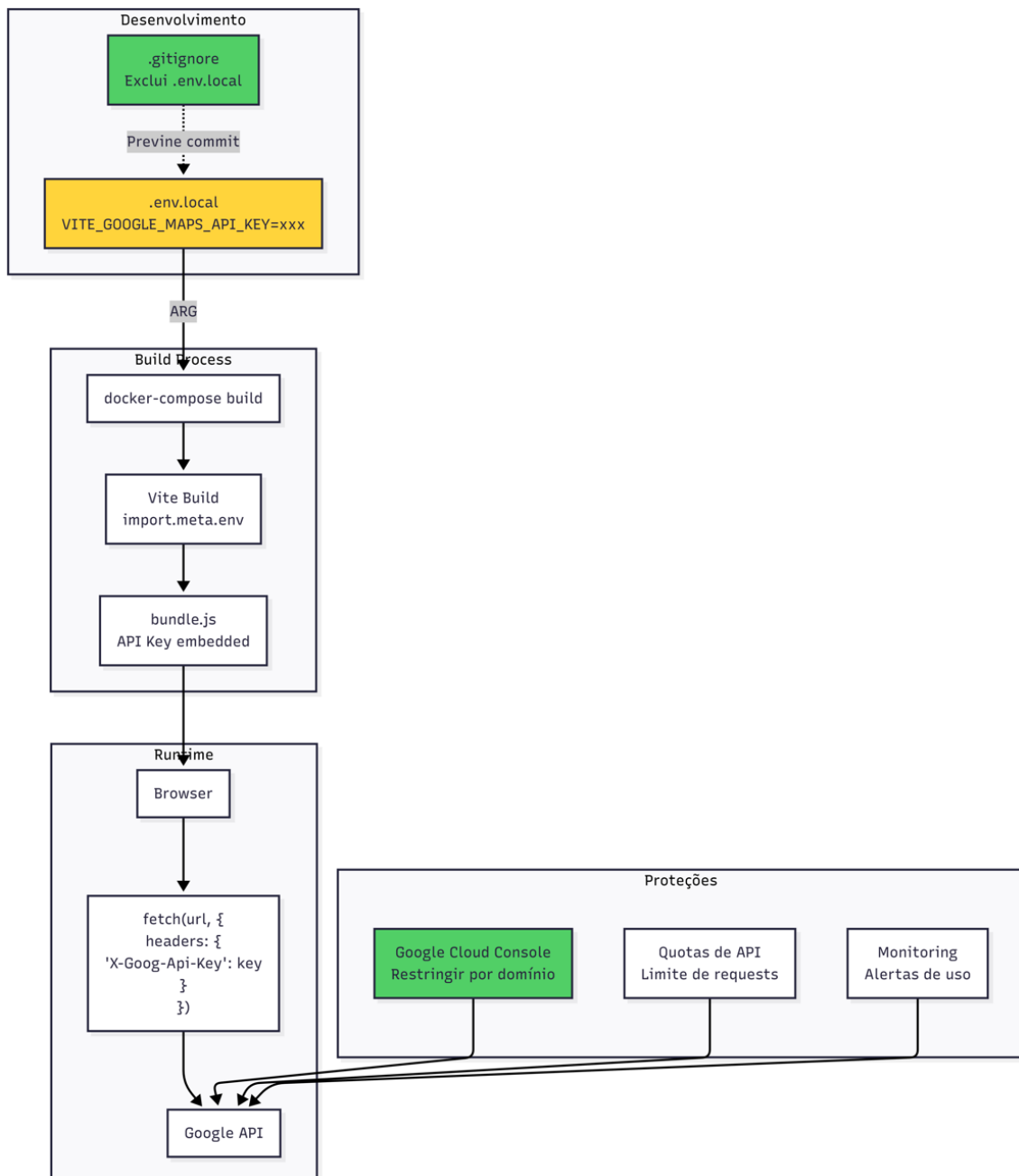


Figura 16 - Fluxo de Segurança das chaves das APIs

A gestão de chaves de API constitui um aspeto crítico da segurança do sistema, em particular devido à utilização de serviços externos que requerem autenticação, como a Google Maps Directions API e Google Places API. No Map

Route Explorer, as API keys são geridas exclusivamente através de variáveis de ambiente, definidas em ficheiros .env (ou .env.local em desenvolvimento) que são explicitamente excluídos do controlo de versão. Esta abordagem evita a exposição accidental de credenciais sensíveis no repositório de código.

Conforme ilustrado no diagrama de gestão de API keys (Figura 16), estas credenciais são injetadas no momento de build da aplicação e incorporadas no bundle JavaScript final. Embora esta decisão implique que a chave esteja tecnicamente acessível no código cliente, tal opção é justificada pela natureza das APIs utilizadas e é mitigada através de mecanismos adicionais. Entre estes mecanismos incluem-se a restrição das chaves por domínio na consola da Google Cloud, a definição de quotas de utilização e a monitorização contínua de padrões de uso, reduzindo o impacto potencial de uma eventual exposição.

8.4 Headers de Segurança

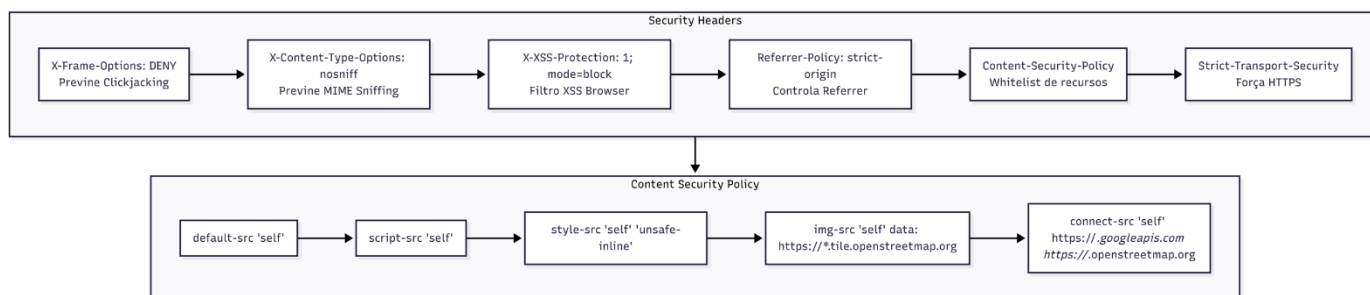


Figura 17 - Configuração de Segurança do Nginx

O sistema implementa um conjunto explícito de security headers ao nível do servidor Nginx, conforme sintetizado no diagrama de Security Headers e Content Security Policy. Estes mecanismos visam mitigar vetores de ataque comuns em aplicações web client-side.

Os headers X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff e X-XSS-Protection: 1; mode=block são utilizados para prevenir, respetivamente, ataques de clickjacking, exploração de MIME sniffing e execução de cross-site scripting suportado pelo browser. O header Referrer-Policy: no-referrer-when-downgrade limita a propagação de informação sensível no header Referer, reduzindo exposição involuntária de URLs e parâmetros.

Complementarmente, é aplicada uma Content Security Policy (CSP) restritiva, que define explicitamente as origens permitidas para carregamento de recursos. A política baseia-se no princípio de default deny, permitindo apenas recursos provenientes do próprio domínio (default-src 'self'). Scripts permitem 'unsafe-inline' e 'unsafe-eval' (necessário para React/Tailwind) e estilos são limitados ao domínio da aplicação, imagens são autorizadas a partir de fontes locais, data:,

blob:, servidores de tiles do OpenStreetMap e domínios Google (para mapas e imagens de APIs)

O header Strict-Transport-Security (HSTS) reforça a obrigatoriedade de comunicações HTTPS, prevenindo ataques de protocol downgrade e garantindo que o browser apenas estabelece ligações seguras com a aplicação.

A combinação destes headers com a CSP, como é possível verificar-se Figura 17, reduz significativamente a superfície de ataque associada à execução de código no cliente, assegurando que apenas recursos explicitamente autorizados podem ser carregados ou executados no contexto da aplicação.

9. Testes e Validação

Este capítulo descreve a estratégia abrangente de testes e validação implementada no Map Route Explorer, com o objetivo de garantir correção funcional, robustez técnica, qualidade da experiência do utilizador e desempenho aceitável do sistema. A abordagem adotada combina testes unitários automatizados, validação de integrações com serviços externos, testes de gestão de estado e análise de performance, refletindo uma estratégia sistemática e alinhada com boas práticas modernas de engenharia de software.

A suite de testes foi desenvolvida utilizando a framework Vitest, integrada no ecossistema Vite e otimizada para projetos React e TypeScript. Esta escolha permite execução rápida, isolamento eficiente de dependências através de mocks e validação consistente e reproduzível das funcionalidades críticas da aplicação. A configuração do ambiente de testes utiliza jsdom para simular APIs do navegador, permitindo testar componentes e hooks que dependem de funcionalidades como geolocalização, manipulação de DOM e criação de blobs.

9.1 Arquitetura de Testes

A arquitetura de testes do Map Route Explorer organiza-se em quatro fases principais, cada uma focada em diferentes camadas e responsabilidades do sistema.

9.1.1 Fase 1: Testes de Serviços de API

A primeira fase concentra-se na validação dos serviços que integram com APIs externas, constituindo a camada de comunicação com serviços de terceiros. Esta fase é crítica pois garante que a aplicação interage corretamente com serviços externos e processa adequadamente as respostas recebidas.

Os testes validam a integração completa com a Google Maps Directions API, incluindo construção correta de DirectionsRequest com diferentes modos de transporte (automóvel, bicicleta, caminhada, transporte público), processamento

de respostas da API com decodificação de polylines, agregação de métricas de múltiplos legs (distância total, duração total), extração e sanitização de instruções HTML para prevenir vulnerabilidades XSS, suporte a waypoints (até 5 parágrafos intermediárias), tratamento de erros (API key inválida, quota excedida, sem rota encontrada), e normalização de geometria para formato LineString. A validação da API key é realizada através de verificação da configuração no ambiente de testes.

Os testes cobrem a integração com a Google Places API para geocodificação primária, validando pesquisa de localização através de AutocompleteService, reverse geocoding utilizando Geocoder, construção correta de requests, parsing de resultados Google Places, tratamento de erros (API key, quota, invalid request) e limitação de resultados conforme parâmetro especificado. Os testes utilizam mocks extensivos do Google Maps SDK para simular o ambiente do navegador.

Os testes validam o serviço de geocodificação fallback, confirmando que a aplicação utiliza Google Places como serviço primário e recorre a Nominatim apenas quando o serviço principal falha. Os cenários testados incluem pesquisa de localização por texto, reverse geocoding, mecanismo de fallback automático, tratamento de erros específicos (418 rate limit, 403 forbidden, timeout), parsing de resultados Nominatim e validação de coordenadas retornadas. Os testes garantem que a aplicação mantém funcionalidade mesmo quando um serviço externo está indisponível.

Os testes validam a integração com OSRM como provedor secundário de routing, incluindo construção correta de URL de rota, suporte a diferentes perfis, processamento de resposta GeoJSON, ajuste de duração baseado em velocidades médias realistas quando a API retorna valores irrealistas, tratamento de erros (sem rota, timeout), e suporte a waypoints respeitando limitações do OSRM.

Os testes validam a integração com Overpass API para obtenção de pontos de interesse, incluindo construção de queries Overpass QL, filtragem por categoria (restaurantes, cafés, postos de combustível, estacionamento, atrações), limitação de resultados (100 POIs), parsing de resposta Overpass, tratamento de timeouts (15s) e filtragem por bounds geográficos com padding adequado.

Os testes validam a integração com o chatbot n8n, cobrindo o envio de mensagens com payload correto, a inclusão de userLocation e useCurrentLocationAsOrigin (se for fornecido), processamento de respostas com ação set_route (origem, destino, waypoints), tratamento de erros específicos (timeout, 404, 500+, erros de rede), verificação de saúde do webhook através de checkN8NHealth, e envio de localizações com ações específicas.

9.1.2 Fase 2: Testes de Utilitários

A segunda fase concentra-se na validação de funções utilitárias que suportam operações comuns em toda a aplicação, garantindo correção matemática, formatação adequada e interação correta com APIs do navegador.

Os testes validam funções de cálculo e formatação de rotas, incluindo cálculo de duração baseado em distância e modo de transporte, ajuste de duração quando valores da API são irrealistas (detecção e correção baseada em velocidades médias), formatação de duração em formato legível (horas, minutos, segundos), e formatação de distância em quilómetros com precisão adequada.

Os testes validam funcionalidades de exportação de rotas, incluindo exportação para formato JSON com estrutura correta e dados completos, exportação para formato GPX com estrutura XML válida e metadados adequados, criação de nomes de ficheiro determinísticos baseados em timestamp e modo de transporte, interação correta com API do navegador (criação de blobs, URLs temporárias, acionamento de download) e limpeza de recursos (revokeObjectURL).

Os testes validam construção de URLs para abertura de rotas no Google Maps, incluindo construção correta de URL com origem e destino, inclusão de waypoints se estiverem presentes, especificação correta do modo de transporte, e codificação adequada dos parâmetros.

Os testes validam funções de cálculo de bounds geográficos, incluindo cálculo de bounding box baseado em centro e zoom, normalização de coordenadas para intervalo válido (-180 a 180 graus para longitude, -85 a 85 graus para latitude), detecção de mudanças significativas em bounds para evitar requisições excessivas e tratamento de ambiente server-side (window undefined).

Os testes validam o hook useGeolocation para obtenção de localização atual, incluindo obtenção bem-sucedida de coordenadas, tratamento de permissão negada pelo utilizador, tratamento de timeout na obtenção de localização, tratamento de erro de posicionamento indisponível, tratamento de geolocalização não suportada pelo navegador, configuração correta de opções (high accuracy, timeout, maximumAge) e gestão de estados de loading e erro.

9.1.3 Fase 3: Testes de Gestão de Estado

A terceira fase valida a consistência e sincronização do estado global da aplicação através das stores Zustand.

Os testes validam a gestão de estado através de múltiplas stores especializadas, incluindo invalidação automática de rota quando origem ou destino são alterados, gestão de waypoints (adição, remoção, reindexação), sincronização entre MapStore e RouteStore, gestão de estado de pesquisa (query, resultados, loading), e prevenção de estados inconsistentes.

Os testes validam a gestão específica de pontos de interesse, incluindo cache de POIs baseado em bounds geográficos, carregamento de POIs para área específica, recuperação de POIs dentro de bounds e gestão eficiente de cache para evitar requisições redundantes.

9.1.4 Fase 4: Testes de Performance

A quarta fase concentra-se na validação de performance de operações críticas, garantindo que o sistema responde dentro de limites aceitáveis.

Os testes validam tempos de execução de funções utilitárias críticas, incluindo cálculo de duração (< 1ms), ajuste de duração (< 1ms), formatação de duração (< 1ms), formatação de distância (< 1ms), processamento eficiente de valores grandes (distâncias e durações), e processamento em batch de múltiplas operações (400 cálculos < 10ms, 2000 formatações < 20ms).

9.2 Estratégia de Mocking

A estratégia de mocking adotada permite isolar completamente os testes de dependências externas, garantindo execução rápida, reprodutibilidade e independência de serviços externos.

9.2.1 Mocking de APIs HTTP

Para serviços que utilizam Axios (OSRM, Nominatim, POI, N8N), os testes utilizam `vi.hoisted()` para criar mocks de `axios.create` e interceptar métodos HTTP (get, post, options). Esta abordagem permite controlar completamente as respostas das APIs, simular diferentes cenários de erro e validar que os serviços constroem corretamente os requests.

9.2.2 Mocking do Google Maps SDK

Para serviços que dependem do Google Maps SDK (`google-maps.service`, `google-places.service`), os testes utilizam `vi.hoisted()` para criar um mock completo do objeto `window.google.maps`, incluindo `DirectionsService`, `AutocompleteService`, `PlacesService`, `Geocoder`, e `Map`. O mock é configurado antes da importação dos serviços, garantindo que os serviços encontram o ambiente necessário durante a inicialização.

9.2.3 Mocking de APIs do Navegador

Para testes que dependem de APIs do navegador (geolocalização, DOM, Blob), os testes utilizam `vi.stubGlobal()` para mockar objetos globais como `navigator.geolocation` e `document`. O ambiente `jsdom` fornecido pelo Vitest simula o DOM, permitindo testar componentes e hooks que manipulam elementos HTML.

9.2.4 Mocking de Variáveis de Ambiente

A configuração da API key do Google Maps é realizada através de `vitest.config.ts`, utilizando “define” para injetar a variável `VITE_GOOGLE_MAPS_API_KEY` no ambiente de testes. Esta abordagem permite validar que os serviços verificam corretamente a presença da API key sem depender de arquivos `.env` externos.

9.3 Cobertura e Métricas

A suite de testes atual é composta por 14 ficheiros de teste, totalizando 133 testes com sucesso, garantindo uma cobertura abrangente dos principais componentes do sistema. Estes testes validam o comportamento de seis serviços de API, incluindo Google Maps Directions, Google Places, Nominatim, OSRM, POI e N8N, assegurando a correta integração com dependências externas. São igualmente cobertos cinco módulos utilitários relacionados com cálculo de rotas, exportação, integração com Google Maps, gestão de limites geográficos e geolocalização. A fiabilidade da gestão de estado é verificada através de testes dedicados a quatro stores Zustand, nomeadamente RouteStore, MapStore, SearchStore e POIStore. Adicionalmente, existe uma suite específica de testes de performance que valida os tempos de execução das operações críticas, garantindo que o sistema cumpre os requisitos de desempenho definidos.

A cobertura abrange todas as funcionalidades críticas da aplicação, desde integrações com serviços externos até funções utilitárias e gestão de estado. Os testes são executados automaticamente no pipeline CI/CD através do GitHub Actions, garantindo que alterações ao código não introduzem regressões.

9.4 Validação de Integrações

Os testes de integração validam que os diferentes módulos colaboram corretamente, garantindo que a aplicação funciona como um sistema coeso.

9.4.1 Integração entre Stores e Serviços

Os testes validam que alterações no estado (por exemplo, definição de origem e destino) desencadeiam corretamente chamadas aos serviços de routing, e que os resultados são refletidos no estado global. A invalidação automática de rotas quando parâmetros críticos são alterados é validada para prevenir inconsistências.

9.4.2 Integração com APIs do Navegador

Os testes de exportação validam a interação completa com APIs do navegador, criando blobs com conteúdo correto, URLs temporárias através de `URL.createObjectURL`, acionam o download através de elementos `<a>` criados dinamicamente, e limpam recursos através

de `URL.revokeObjectURL`. A utilização de timestamps determinísticos (mockados) garante reprodutibilidade dos testes.

9.4.3 Integração com Serviços Externos

Embora os testes utilizem mocks para isolar dependências externas, a validação da construção de requests e processamento de respostas garante que a aplicação interage corretamente com serviços reais quando em produção. Os testes validam formatos de dados, codificação de parâmetros, tratamento de erros HTTP, e parsing de respostas em diferentes formatos (JSON, GeoJSON, XML).

9.5 Tratamento de Erros

Os testes validam extensivamente o tratamento de erros em diferentes cenários, garantindo que a aplicação fornece feedback adequado e mantém estabilidade mesmo quando serviços externos falham.

9.5.1 Erros de Rede

Os testes simulam diferentes tipos de erros de rede (timeout, conexão recusada, erro genérico) e validam que os serviços retornam mensagens de erro apropriadas e não quebram a aplicação.

9.5.2 Erros de API

Os testes validam tratamento de erros específicos de APIs (404, 500+, quota excedida, API key inválida) e confirmam que a aplicação utiliza mecanismos de fallback quando disponíveis (por exemplo, Nominatim quando Google Places falha, OSRM quando Google Maps falha).

9.5.3 Erros de Validação

Os testes validam que a aplicação detecta e trata dados inválidos (coordenadas fora de limites, rotas com distância/duração zero, respostas vazias) e fornece mensagens de erro claras ao utilizador.

9.6 Testes de Performance

Os testes de performance validam que operações críticas completam dentro de limites aceitáveis, garantindo que a aplicação mantém responsividade mesmo em cenários exigentes.

9.6.1 Performance de Funções Utilitárias

Os testes validam que funções de cálculo e formatação completam em menos de 1ms, garantindo que não introduzem latência perceptível na interface.

9.6.2 Performance em Batch

Os testes validam que processamento em batch de múltiplas operações (centenas de cálculos, milhares de formatações) completa em tempos

aceitáveis, garantindo que a aplicação pode lidar com volumes elevados de dados sem degradação significativa.

9.7 Execução e Integração Contínua

A suite de testes é executada automaticamente no pipeline CI/CD através do GitHub Actions, garantindo que todas as alterações ao código são validadas antes de serem integradas na base principal. Os testes são executados em ambiente Linux (ubuntu-latest) com Node.js 20, garantindo consistência entre ambientes de desenvolvimento, CI e produção.

A execução local dos testes é realizada através do comando `npm test`, que suporta modo `watch` para desenvolvimento iterativo e modo `coverage` para análise de cobertura de código. A configuração do Vitest permite execução paralela de testes, otimizando o tempo total de execução da suite.

10. Integrações Externas

10.1 Google Maps Directions API

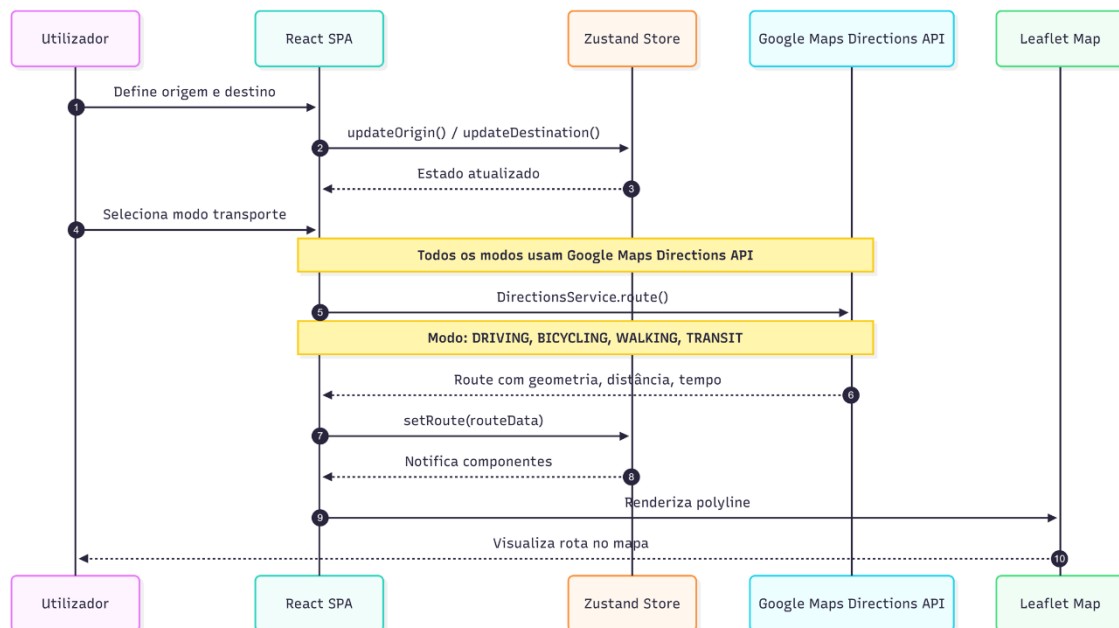


Figura 18 - Fluxo de Cálculo de Rota

A Google Maps Directions API é utilizada na aplicação Map Route Explorer como o serviço principal de cálculo de rotas, sobretudo nos cenários que exigem maior precisão, suporte a transporte público ou consideração de condições de tráfego. A integração é realizada através do carregamento dinâmico do SDK JavaScript da Google, permitindo que a biblioteca seja apenas inicializada quando necessária, reduzindo assim o impacto no carregamento inicial da aplicação. Os pedidos de cálculo de rota são construídos a partir dos dados definidos pelo utilizador, incluindo origem, destino, modo de transporte e eventuais pontos intermédios, sendo enviados para a API utilizando os parâmetros adequados ao contexto de navegação. As respostas fornecidas incluem a geometria detalhada do percurso, estimativas de distância e duração, bem como instruções de navegação, que são posteriormente processadas e convertidas para uma representação interna comum. Esta normalização permite que a aplicação trate de forma uniforme rotas provenientes de diferentes fornecedores, garantindo consistência na visualização e na lógica de negócio.

10.2 OSRM API

O OSRM (Open Source Routing Machine) está como alternativa open-source ao serviço da Google, mas não é utilizado automaticamente como fallback no fluxo principal da aplicação. Esta integração garante maior resiliência à aplicação, permitindo a continuação do funcionamento básico em cenários de

indisponibilidade de serviços proprietários. O OSRM é consumido através de uma interface REST simples, na qual as coordenadas são incluídas diretamente no path do pedido e os resultados são devolvidos em formato GeoJSON. A geometria da rota é extraída diretamente deste formato, simplificando o processo de renderização no mapa. No entanto, devido às limitações do serviço, nomeadamente a ausência de dados de tráfego em tempo real, as estimativas de duração são ajustadas internamente com base em velocidades médias realistas para cada modo de transporte. Esta abordagem permite obter resultados mais coerentes com a experiência real do utilizador, mantendo simultaneamente a independência face a serviços proprietários.

10.3 Google Places API

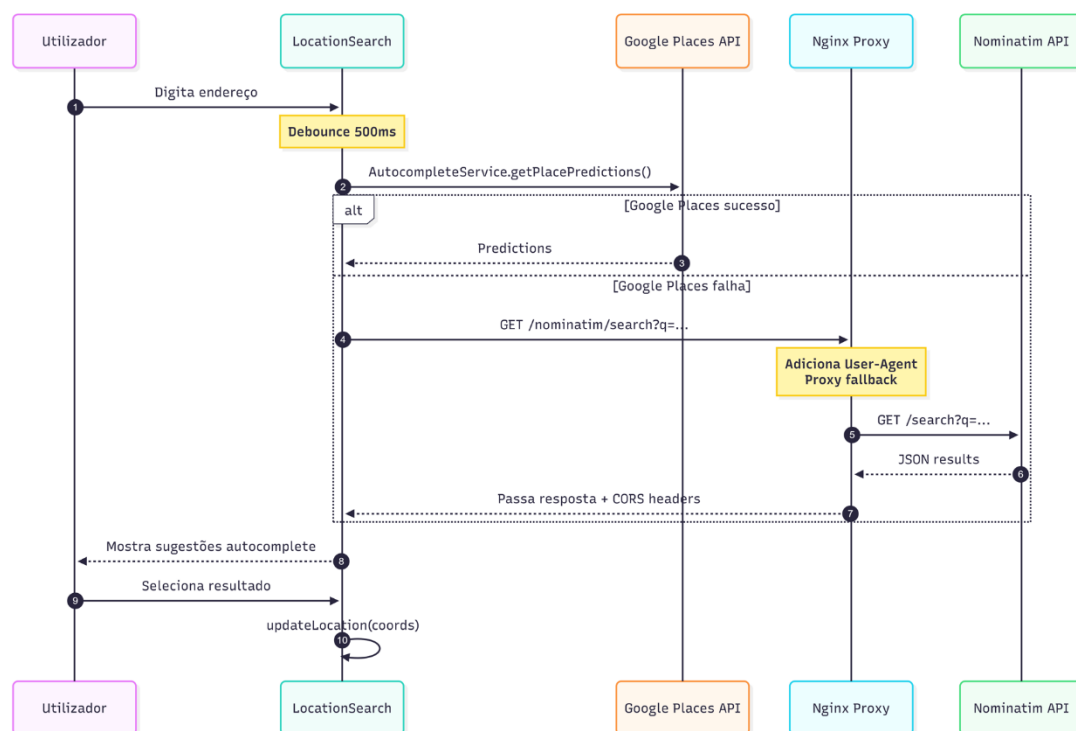


Figura 19 - Fluxo de Pesquisa de Localização

A funcionalidade de pesquisa de locais e endereços baseia-se na integração com a Google Places API como serviço primário, com a Nominatim API como fallback, serviços de geocodificação suportados por dados da Google e do OpenStreetMap respetivamente. Estes serviços permitem converter texto introduzido pelo utilizador em coordenadas geográficas, suportando mecanismos de pesquisa assistida e autocomplete. Para garantir conformidade com as políticas de utilização do serviço e aumentar a robustez da aplicação, os pedidos à Nominatim (quando usada como fallback) não são realizados diretamente pelo frontend, mas encaminhados através de um proxy Nginx. Os pedidos à Google Places API são realizados diretamente pelo frontend. Este proxy é responsável por adicionar cabeçalhos obrigatórios, gerir políticas de CORS e aplicar mecanismos de limitação de pedidos. Do lado da interface, é

utilizado um mecanismo de debounce que reduz o número de chamadas efetuadas durante a introdução de texto, melhorando o desempenho global e evitando sobrecarga desnecessária do serviço externo. Os resultados devolvidos são apresentados ao utilizador sob a forma de sugestões selecionáveis, permitindo uma definição rápida e intuitiva de pontos no mapa.

10.4 Overpass API

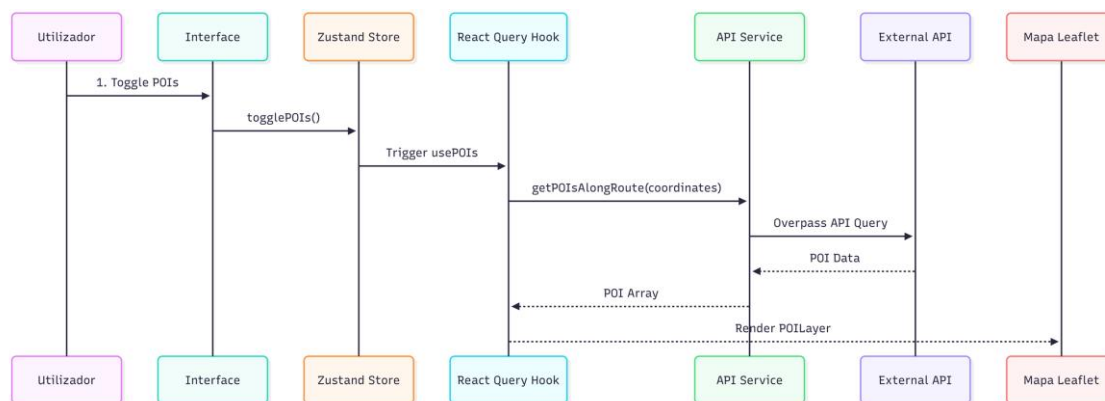


Figura 20 - Fluxo dos pontos de interesse

A Overpass API é utilizada para obter pontos de interesse relevantes ao longo de uma rota ou na área atualmente visível no mapa, enriquecendo o contexto da navegação. Este serviço permite executar consultas avançadas sobre os dados do OpenStreetMap, filtrando elementos com base em categorias específicas, como restauração, combustível ou atrações turísticas. As queries são construídas dinamicamente de acordo com os limites geográficos definidos pela vista atual do mapa ou pela geometria da rota calculada. Para minimizar o impacto no desempenho e evitar pedidos redundantes, a aplicação implementa um sistema de caching que mantém registo das áreas já consultadas, recorrendo a uma store dedicada para gerir os dados obtidos. Os pontos de interesse recolhidos são posteriormente renderizados como marcadores discretos no mapa, permitindo ao utilizador identificar rapidamente serviços relevantes sem comprometer a clareza visual da rota principal.

10.5 n8n Workflow

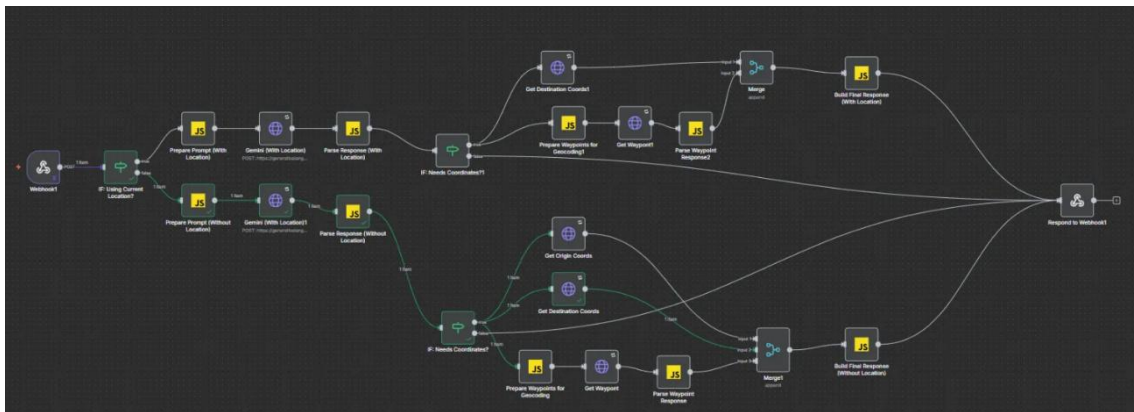


Figura 21 - Fluxo implementado para o chatbot

A aplicação integra ainda um workflow desenvolvido em n8n para suportar funcionalidades de automação e assistência ao utilizador, nomeadamente através de um chatbot. O n8n atua como camada intermédia entre o frontend e serviços externos de inteligência artificial, sendo responsável por receber pedidos, validar dados, enriquecer mensagens com contexto relevante da aplicação e orquestrar chamadas subsequentes. Esta separação permite desacoplar a lógica conversacional da interface principal, facilitando a manutenção e evolução da funcionalidade sem impacto direto no código da aplicação React. A utilização de workflows configuráveis oferece maior flexibilidade, permitindo adaptar ou expandir o comportamento do sistema através de alterações de configuração em vez de desenvolvimento adicional. Esta abordagem contribui para uma arquitetura mais modular e extensível, especialmente em funcionalidades que dependem fortemente de integrações externas.

11. Conclusão

O projeto Map Route Explorer consolidou-se como uma solução de mapeamento robusta e escalável, integrando com sucesso a precisão da Google Maps API com a flexibilidade do OpenStreetMap. Ao longo do desenvolvimento, foi priorizada a criação de uma ferramenta que não calculasse apenas rotas precisas para diversos modos de transporte, como também enriquecesse a experiência do utilizador através da exibição inteligente de pontos de interesse e de um assistente virtual interativo. A aplicação cumpre todos os requisitos funcionais propostos, apresentando uma interface moderna, responsiva e pronta para ser utilizada em cenários reais de exploração geográfica.

A adoção de uma arquitetura em camadas e a implementação de táticas avançadas de gestão de estado e performance, revelou-se um pilar fundamental para assegurar a eficiência operacional da aplicação. O uso do React Query para caching de dados externos permitiu um controlo rigoroso dos custos das APIs pagas, enquanto a integração do n8n como middleware para o chatbot demonstrou como o desacoplamento de serviços pode aumentar a flexibilidade do sistema. A infraestrutura baseada em Docker e Nginx garantiu que o ciclo de vida da aplicação fosse completo, desde o desenvolvimento modular até ao deployment automatizado e seguro em ambiente de produção.

Por fim, a realização deste trabalho foi uma mais-valia, uma vez que, nos proporcionou uma aprendizagem fundamental sobre como converter conceitos teóricos de arquitetura e desenho de software em decisões técnicas pragmáticas. Aprendemos que o desenho de um sistema deve ter em conta os atributos de qualidade, como a manutenibilidade e a interoperabilidade, e que a escolha de tecnologias deve estar sempre alinhada com a sustentabilidade do projeto.